

先進計算機構成論 05

東京大学大学院 情報理工学系研究科 創造情報学専攻

塩谷 亮太

shioya@ci.i.u-tokyo.ac.jp

前回の内容

- パイプライン
- 各種のハザードと解消方法
 - 構造ハザード
 - 非構造ハザード
 - データ・ハザード
 - 制御ハザード

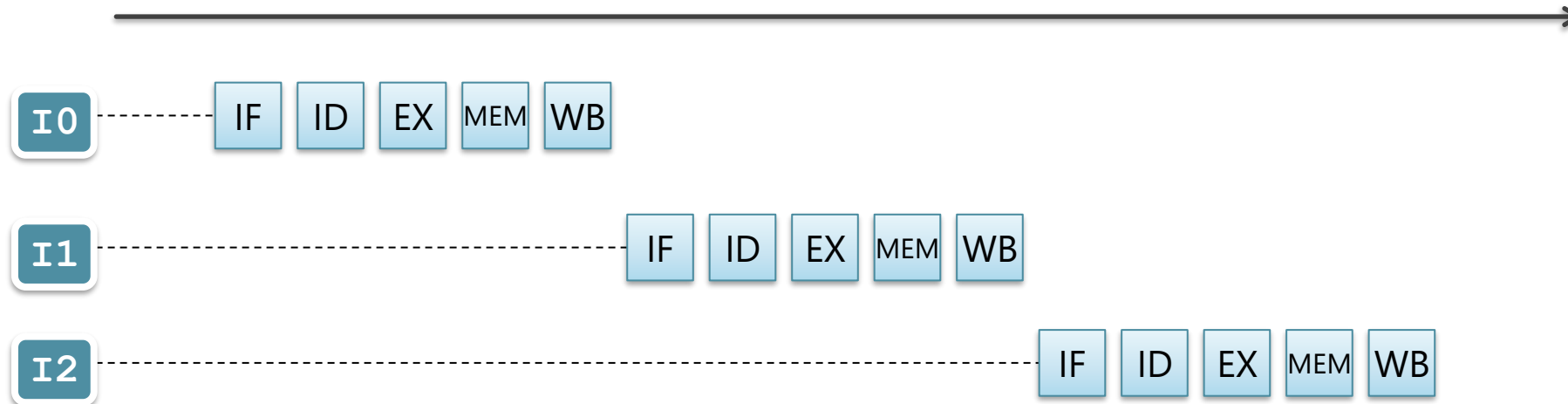
今日の内容

1. 命令パイプラインと性能
2. 分岐予測の基本

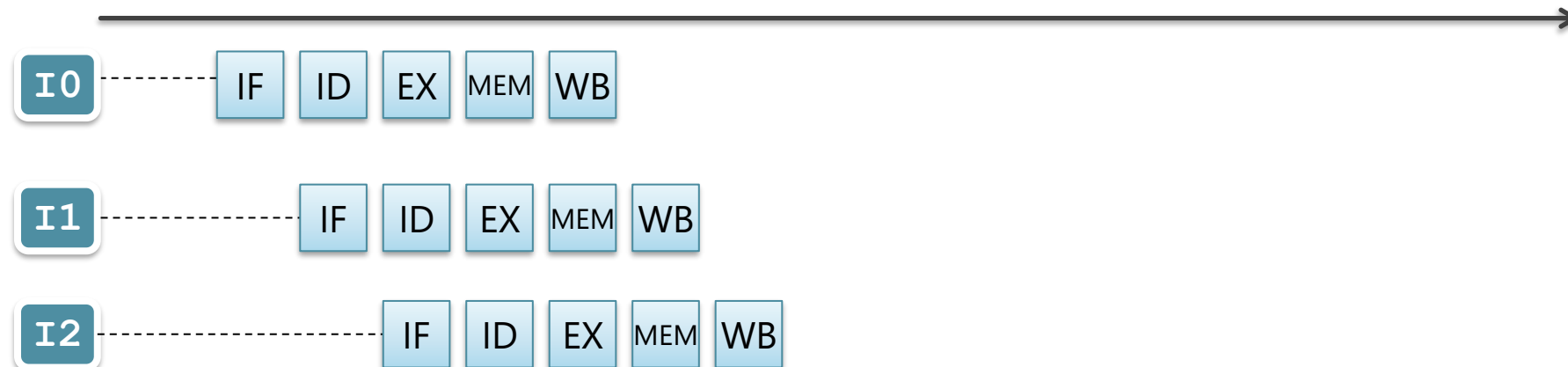
命令パイプラインと性能

パイプライン化によるスループット向上

パイプライン化しない場合



パイプライン化した場合



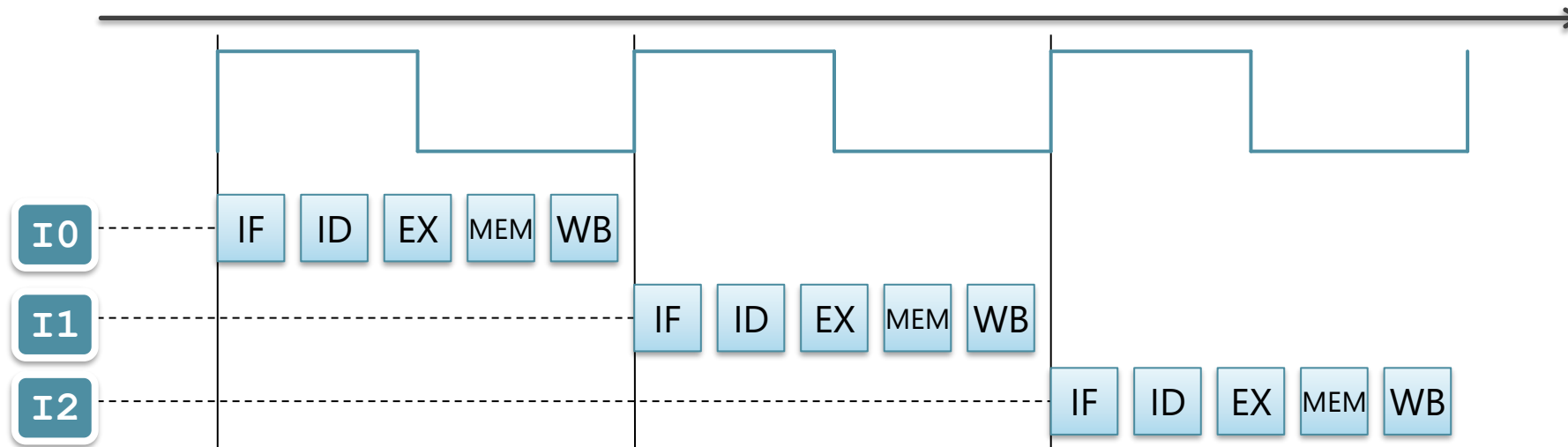
パイプライン化の意味

- パイプライン化の効果：
 - スループットの向上
 - = 単位時間あたりに処理できる命令の数の増加
 - = 動作クロック周波数の向上

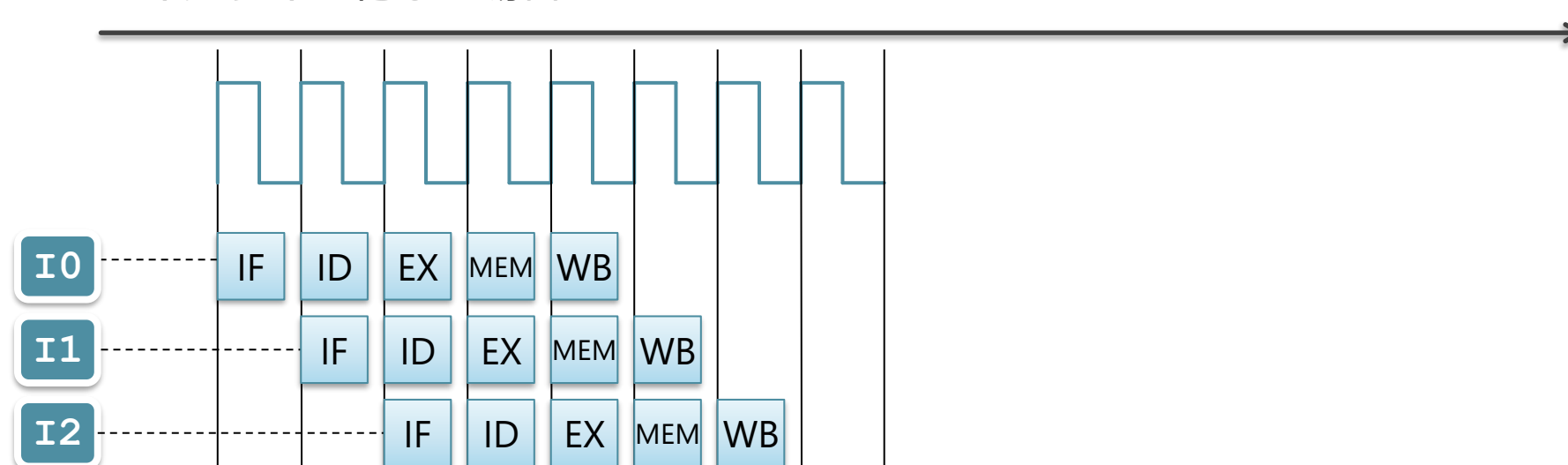
パイプライン化によるクロック周期の短縮

クロックの立ち上がりごとに、1命令が処理

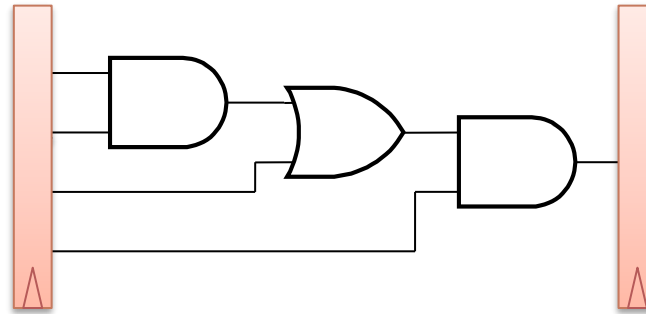
パイプライン化しない場合



パイプライン化した場合



ステージ内の信号の伝播を考える



■ パイプライン：ステージ：

- 複数のパイプライン・ラッチで挟まれてた,
- 組み合わせ回路（ゲート）

■ 矢印の動き：

- クロック開始時に、左のラッチからでた信号が
- 組み合わせ回路を通して、伝播していく様子

2 段にパイプライン化した場合

パイプライン化せず



2 段パイプライン



- クロック周波数は 2 倍に：
 - 各矢印の伸びる速度（信号が伝播する速度）自体は同じ
 - 2 段パイプラインでは、ラッチから 2 回信号が出ている

4段にパイプライン化した場合

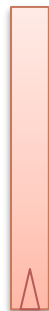
パイプライン化せず



2 段パイプライン



4 段パイプライン



■ クロックが4 倍に

- 4 段パイプラインでは, ラッチから 4 回信号が出ている

パイプライン化の限界



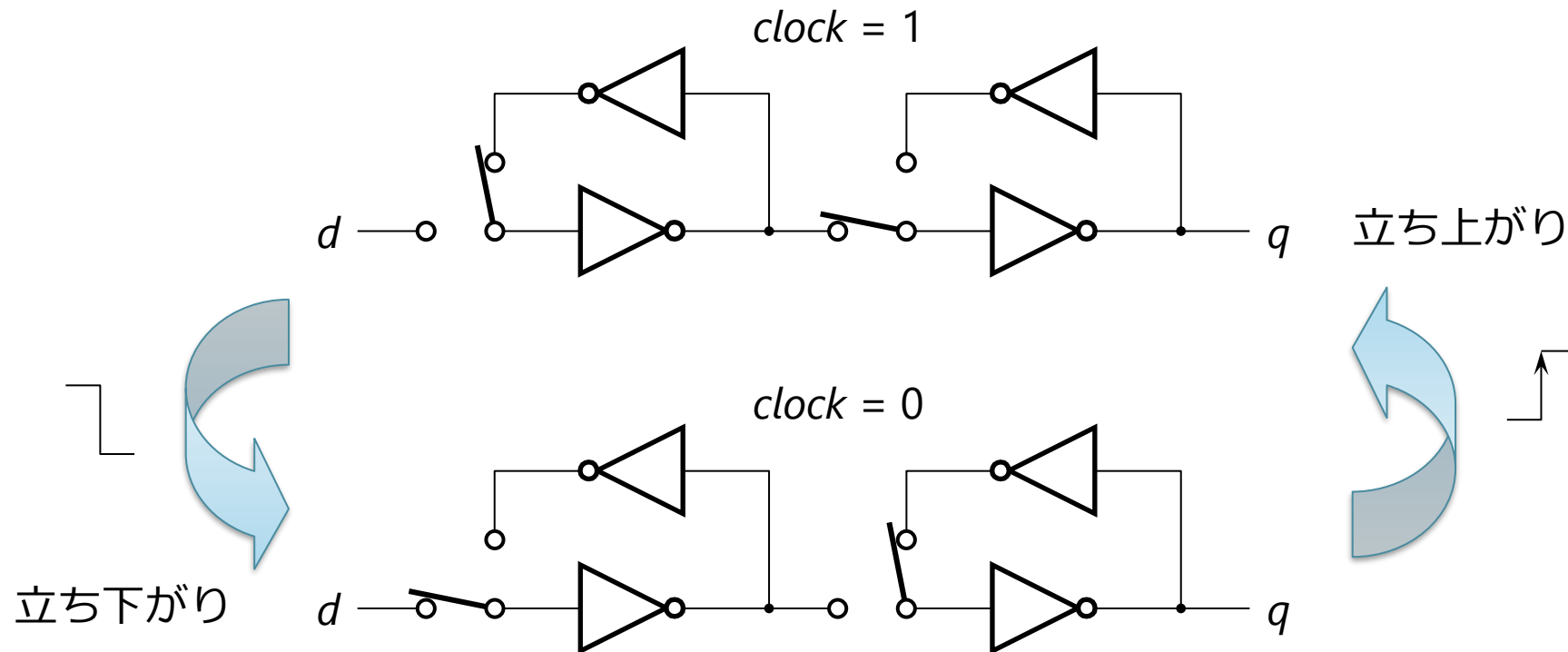
- パイプライン段数を増やしていけば、どこまでも速くなるのか？
 - ならない
- 理由：
 1. 回路的な理由による周波数向上の限界
 2. アーキテクチャ的な理由による実効性能の限界

■ 理由：

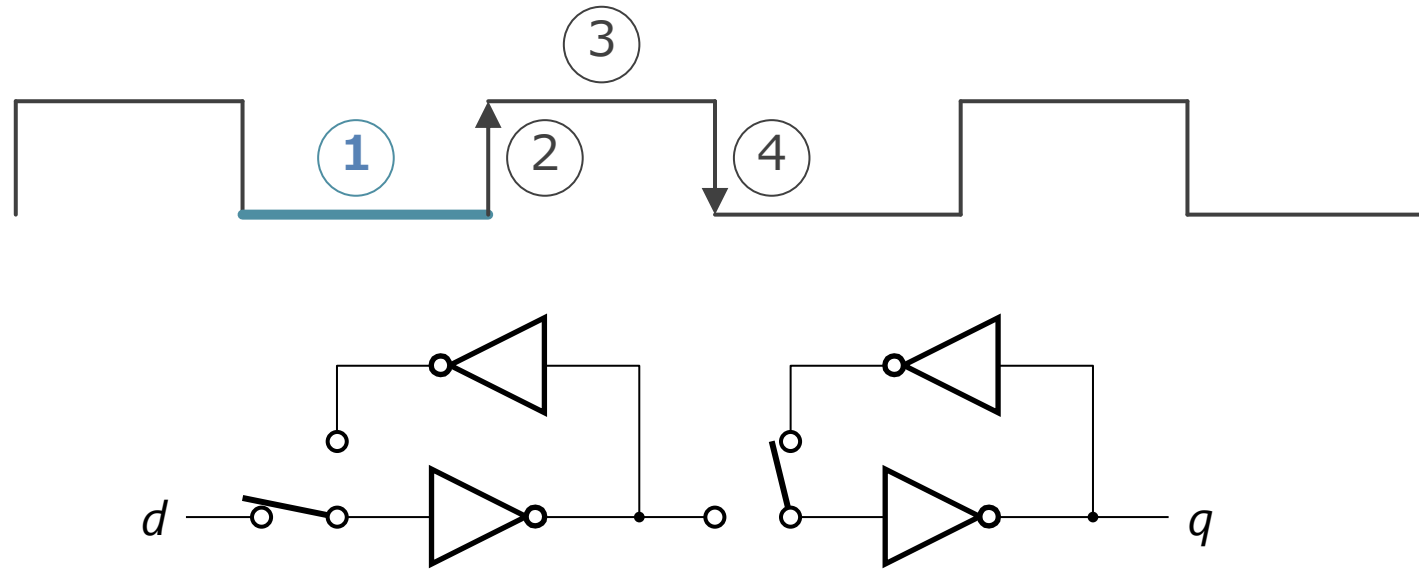
1. D-FF 自体の遅延のため
2. 消費電力と熱のため

D-FF の回路

- 構造：マルチプレクサが入った2つのインバータのループ
 - ◇ マルチプレクサを，切り替えスイッチとして説明
 - ◇ クロックの立ち上がりのたびに， d の値がサンプリングされる
 - ◇ その値が次のサイクルの間 q から出力される



D-FF の動作 ① クロック信号が Low にあるとき



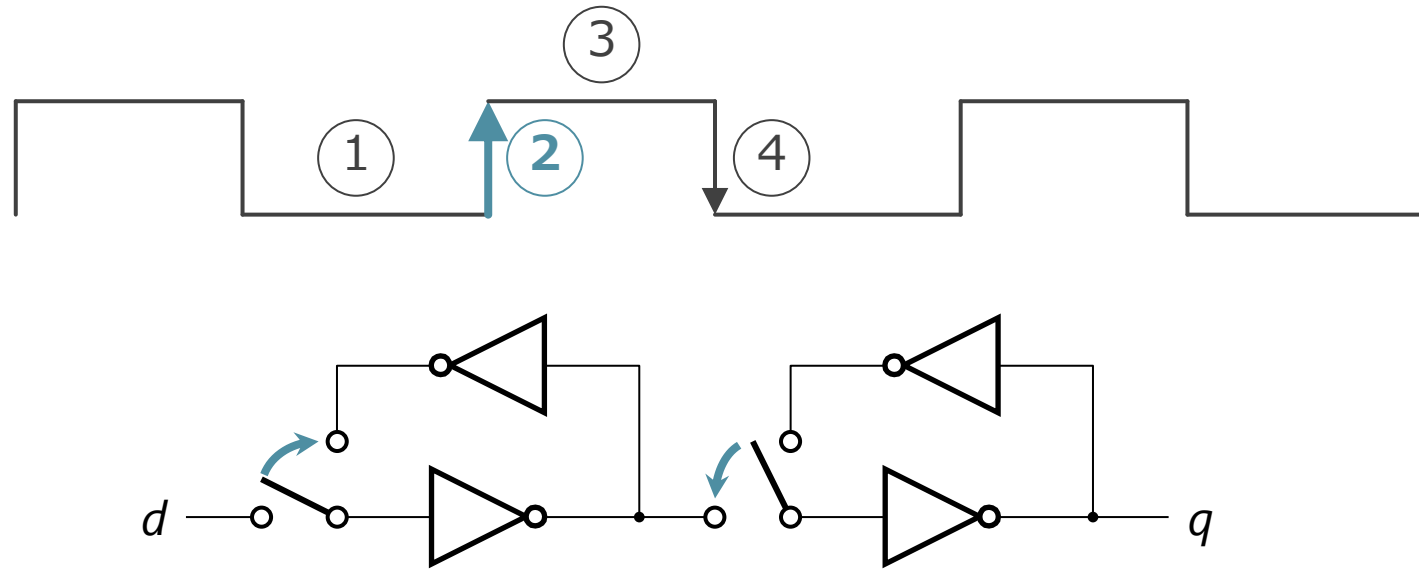
■ 左側のループ：

- ◇ d の入力の変化に応じて、インバータの状態が随時切り替わる
- ◇ 右側のループとは遮断されている

■ 右側のループ：

- ◇ ループのインバータの状態（=記憶）が q に出力され続ける

D-FF の動作 ② クロック信号の立ち上がり



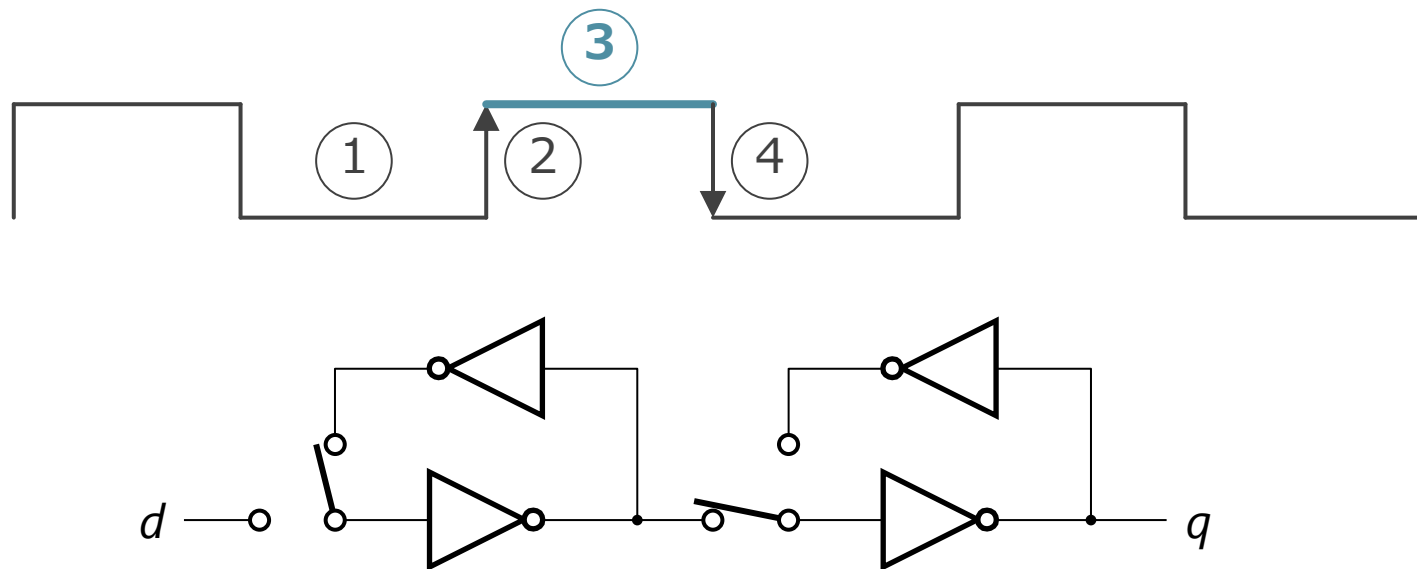
■ 左側のループ :

- ◇ d と遮断され, ループが形成される
- ◇ 直前まで d に入力されていた信号が記憶される

■ 右側のループ :

- ◇ 左側のループと繋がり, ループが解除される

D-FF の動作 ③ クロック信号が High



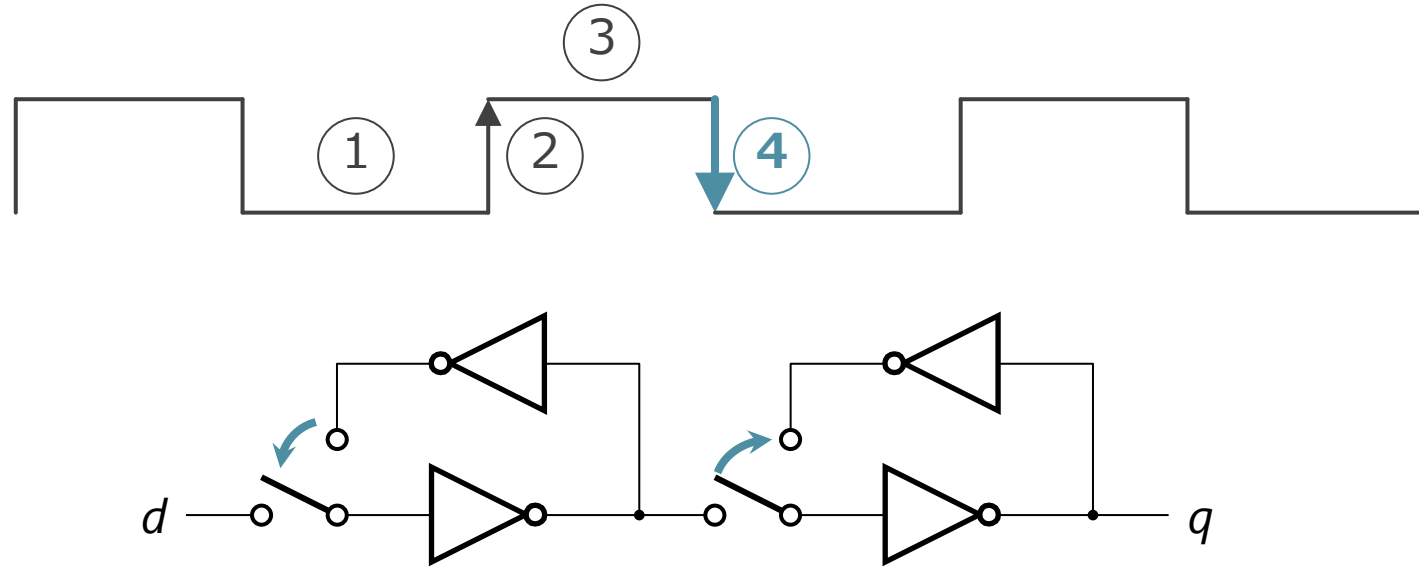
■ 左側のループ：

- ◇ クロックが立ち上がる直前の d の内容を出し続ける

■ 右側のループ：

- ◇ 左側のループの出力を反転して q に出力

D-FF の動作 ④ クロック信号の立ち下がり



■ 左側のループ :

- ◇ ループが解除される

■ 右側のループ :

- ◇ 左側のループと遮断され, ループが形成される
- ◇ それまで左側から入力された内容を出し続ける

D-FF の遅延

- D-FF の遅延：これまでの4フェーズの動作の遅延
 - スイッチが切り替わるまでの遅延
 - スイッチが切り替わった後,
インバータの入力に応じて出力が変化するまでの遅延
- クロック周波数を上げすぎると、これらの限界にぶつかる
 - 1ステージ内の組み合わせ回路の遅延：
インバータ換算で通常10から20段分ぐらい
 - なので、D-FF 自体の遅延は意外とバカにならない

理由2：消費電力と熱

■ クロック周波数を上げる

- → 単位時間あたりの回路全体の充放電の回数が増える
- → 消費電力と、それによって発生する熱がそれだけ増える

1. 電力供給の限界

- CPU のチップの端子から流し込める電流の限界
- オームの法則： $V=IR$
 - 端子のピンの数で R が決まる

2. 放熱の限界

- 温度の上昇に、放熱が追いつかない

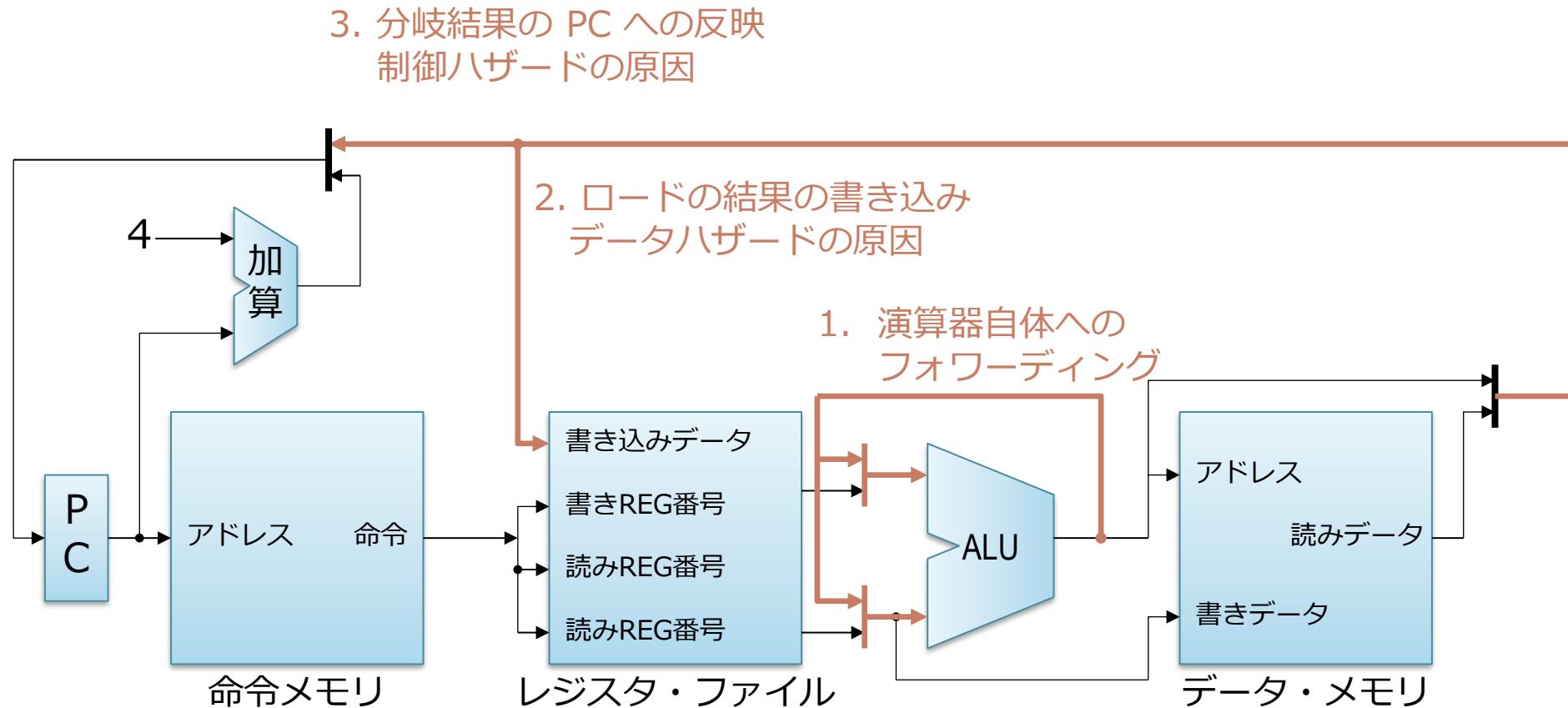
パイプライン化の限界

- 速度が上がらなくなる理由：
 1. 回路的な理由による周波数向上の限界
 2. **アーキテクチャ的な理由による実効性能の限界**

アーキテクチャ的な理由による実効性能の限界

- バックエッジがないパイプライン
 - （回路的な限界にあたるまでは
 - パイプライン段数を増やせば増やすほど性能（周波数）が上がる
- バックエッジがあるパイプライン
 - パイプライン段数を増やすと,
 - 周波数そのものは上がる・・・が,
 - 場合によって, 命令を処理できる実効的な速度が落ちる

バックエッジ：逆方向（右から左）にいく信号

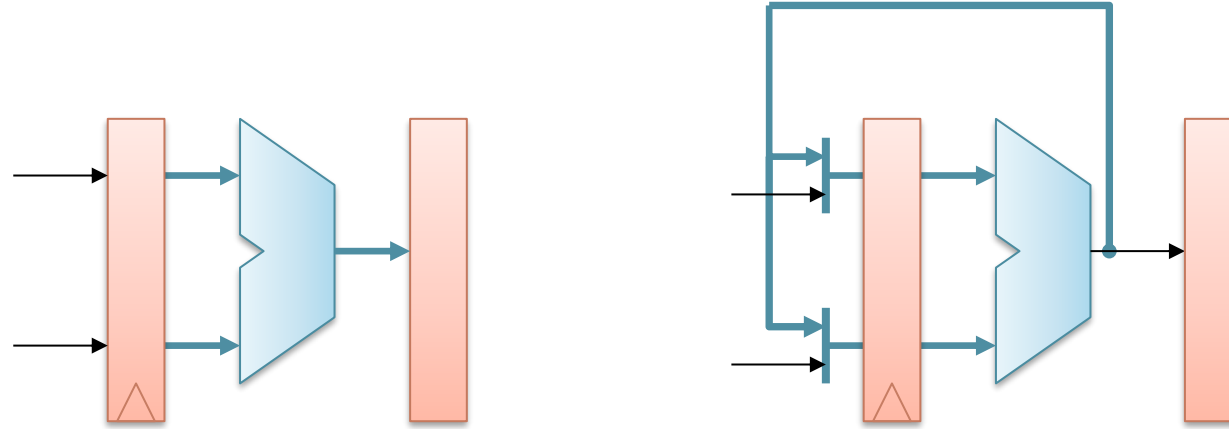


- バックエッジがあるため、命令を単純に流せない場合がある
 - 工場のラインのように、一方向に流せない

問題となるバックエッジ

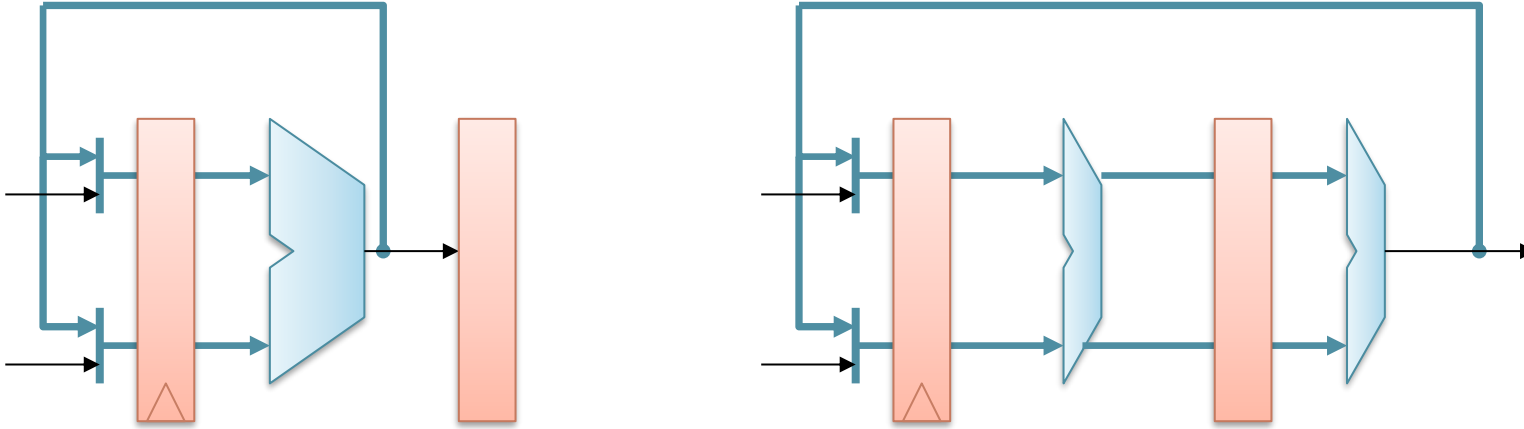
1. 演算器のフォワーディング
2. ロードによるデータ・メモリの読み出し
3. 分岐結果の PC への反映

1. 演算器のフォワーディング



- ここは結構遅延が長い
 - クロック周波数の低下につながるのでパイプライン化したくなる

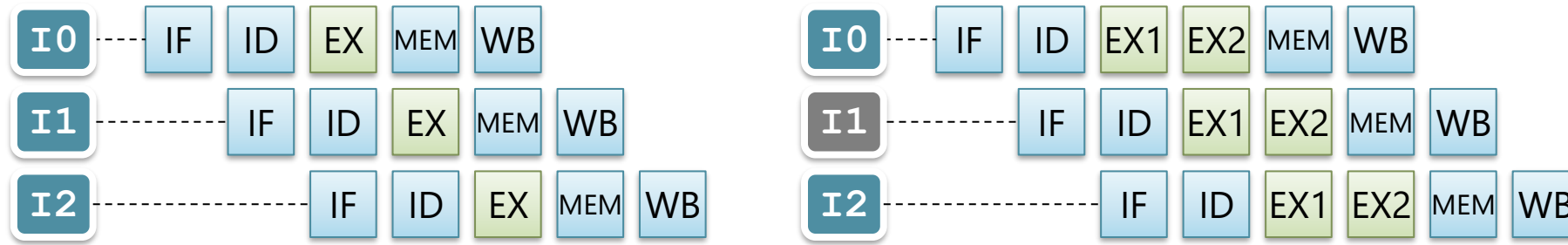
演算器のパイプライン化



■ 演算器のパイプライン化

- たとえば加算を, 下位 32 ビットと上位 32 ビットを 2 ステージかけてやる
- 1 ステージあたりの遅延は半分になる

演算器をパイプライン化した場合の問題

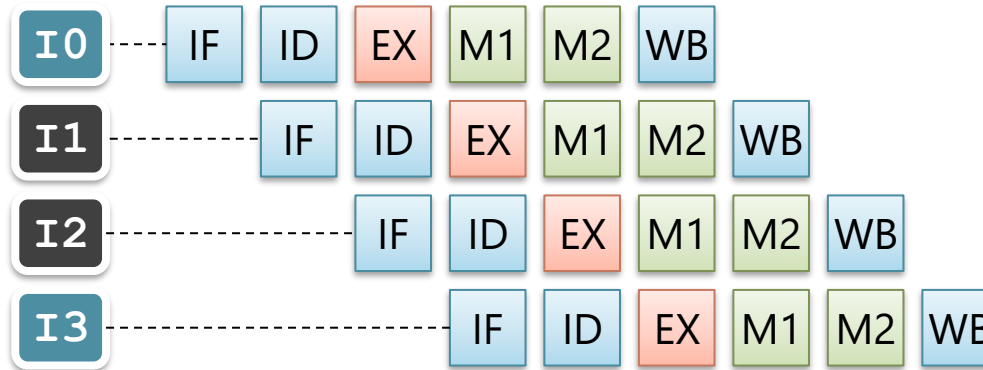


- 依存関係にある命令を連続して実行できなくなる
 - I0 の EX2 が終わる前に, I1 の EX1 が始まる
 - もし, 他に実行すべき命令がおけなければ, 遊ばせとくしかない
- 場合によっては性能が返って下がる
 - 周波数が上がったが, 2 サイクルに 1 回しか命令が実行できない
- 基本的な整数演算はパイプライン化せず 1 ステージを死守するのが普通
 - 乗除算や, 浮動小数点演算はあきらめてパイプライン化

問題となるバックエッジ

1. 演算器のフォワーディング
2. **ロードによるデータ・メモリの読み出し**
3. 分岐結果の PC への反映

ロードによるデータ・メモリの読み出し



■ 演算器の場合とほぼ同様

- 上は, MEM が M1 と M2 にパイプライン化された場合
 - I1 と I2 は, I0 の結果を使えない
- MEM ステージをパイプライン化すると, この部分が長くなる

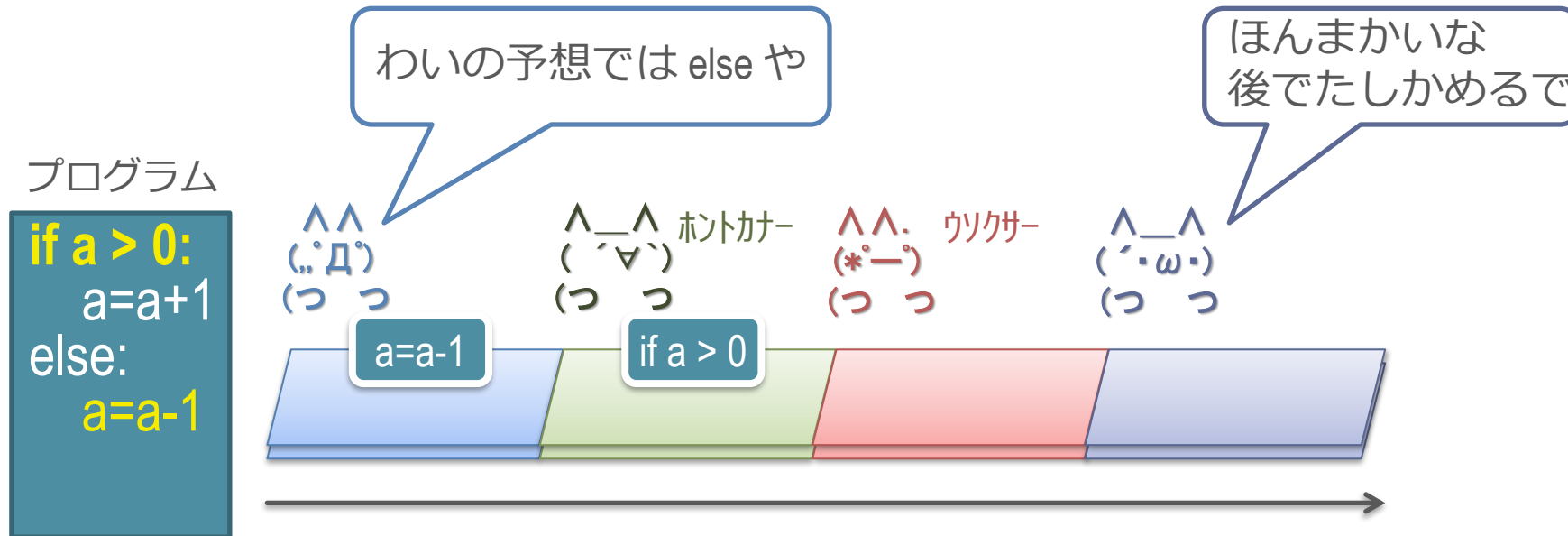
■ しかし, この部分をパイプライン化することはよくある

- ロードは演算よりは出現頻度が低い
- メモリ（キャッシュ）のレイテンシは演算器よりかなり長くなること
が多いためしかたない

問題となるバックエッジ

1. 演算器のフォワーディング
2. ロードによるデータ・メモリの読み出し
- 3. 分岐結果の PC への反映**

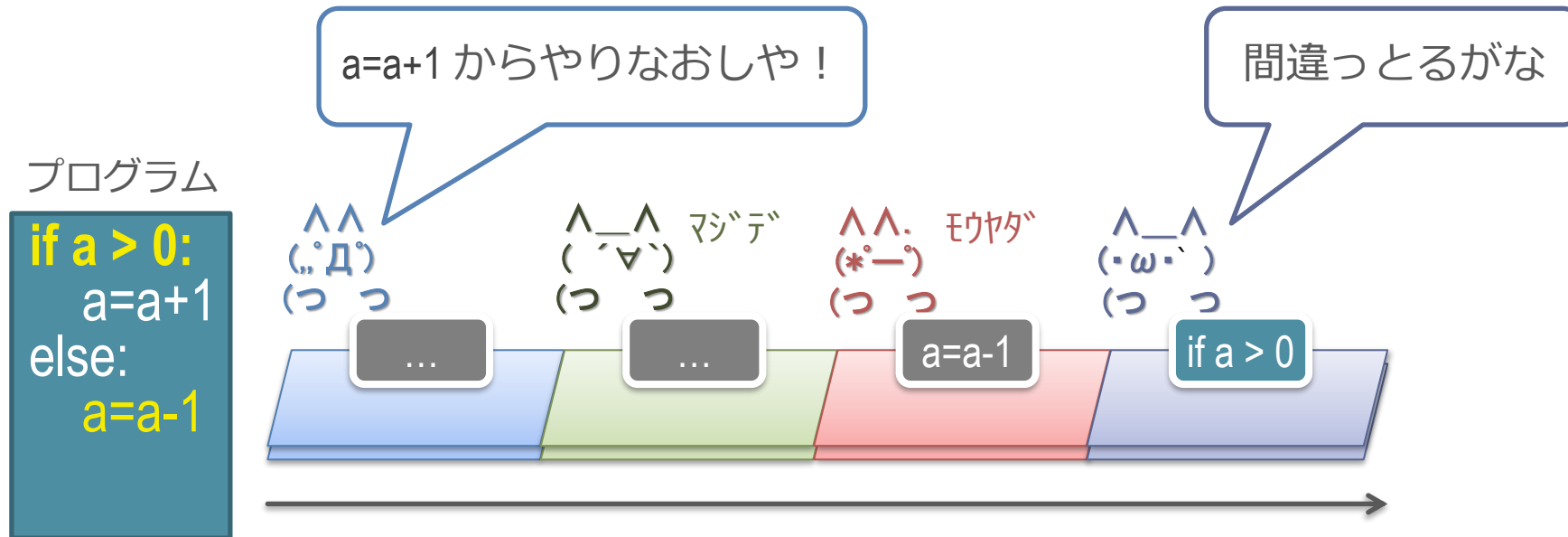
分岐予測



■ 動作

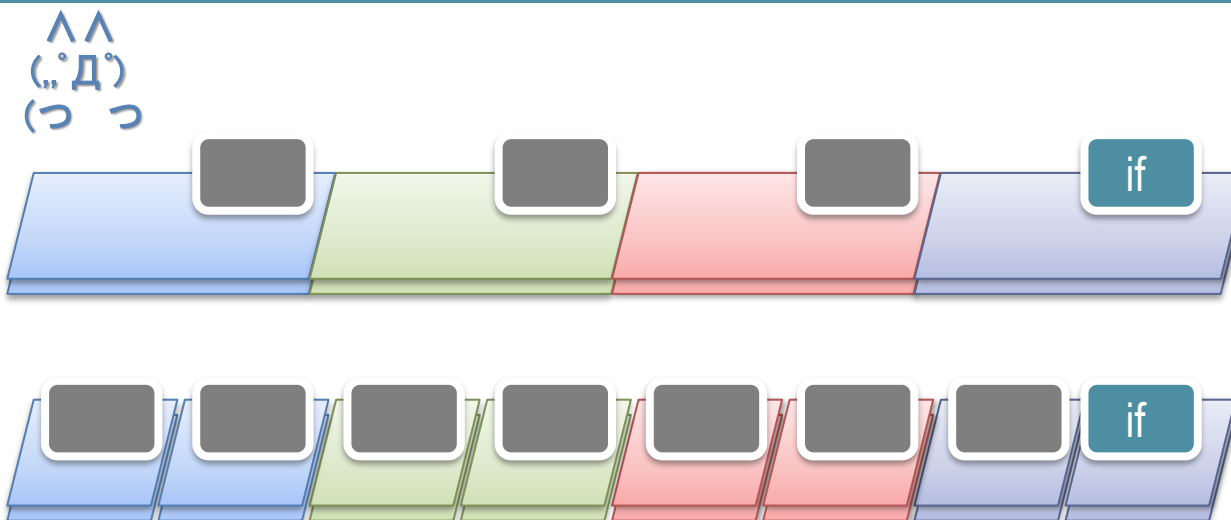
- 「if a > 0」の結果を予測して、命令を取り込む
 - 前回はこっちに行ったので、次もこっちに違いないとかで予測
- あとから予測が正しいか確認する

分岐予測ペナルティ



- 予測が間違っていた場合，以降の処理を取り消してやり直す
- この図では，無駄になるのは3命令分

分岐予測ペナルティの大きさ



- パイプラインを深くすると,
 - = if が右に到達してミスが判明するまでのステージが増える
 - = 予測ミス時に取り消される命令数が大きくなる
 - 一瞬で全員を消せず, 取り消す命令数に応じた時間がかかる
- 実時間が伸びているわけではないことに注意
 - if が右に到達するまでの実時間は変わってない
 - 矢印が伸びるアニメーションを思い出してほしい

パイプライン化の限界のまとめ

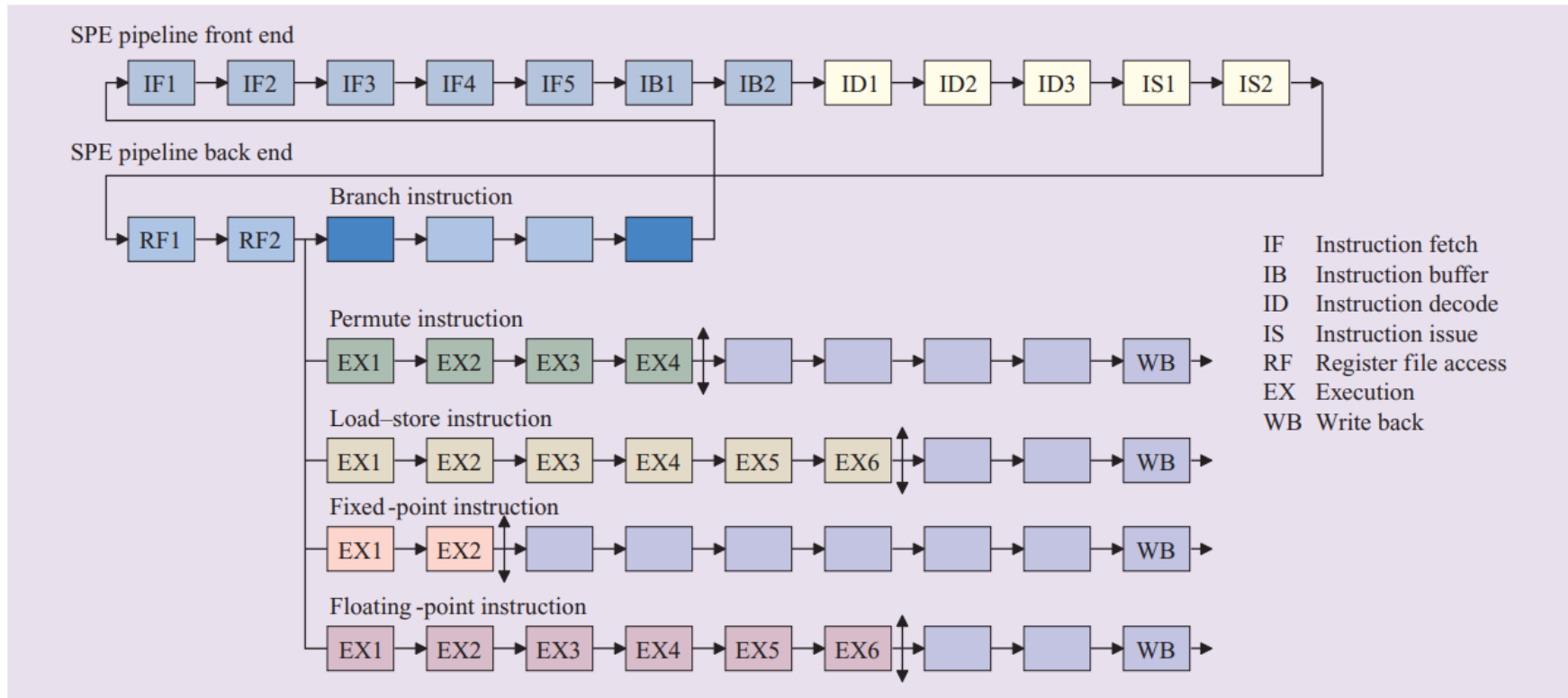
- 速度が上がらなくなる理由：
 - 回路的な理由による周波数向上の限界
 - D-FF の遅延
 - 電力と熱
 - アーキテクチャ的な理由による実効性能の限界
 - バックエッジによる実効性能の低下
 - （今日話した話題意外に、スーパスカラ固有の性能低下も

実際の CPU のパイプライン段数

- 現在は大体 15 ～ 20 段
- Intel Pentium4 (Prescott) 31 段
 - 2004年発売で 3.8 GHz
 - おそらく, 歴史上最大の段数
 - 熱くなりすぎ & 性能が出ずで, この後ステージ数は減少
- AMD Zen : 19 段
 - 2017年発売で 4.2GHz

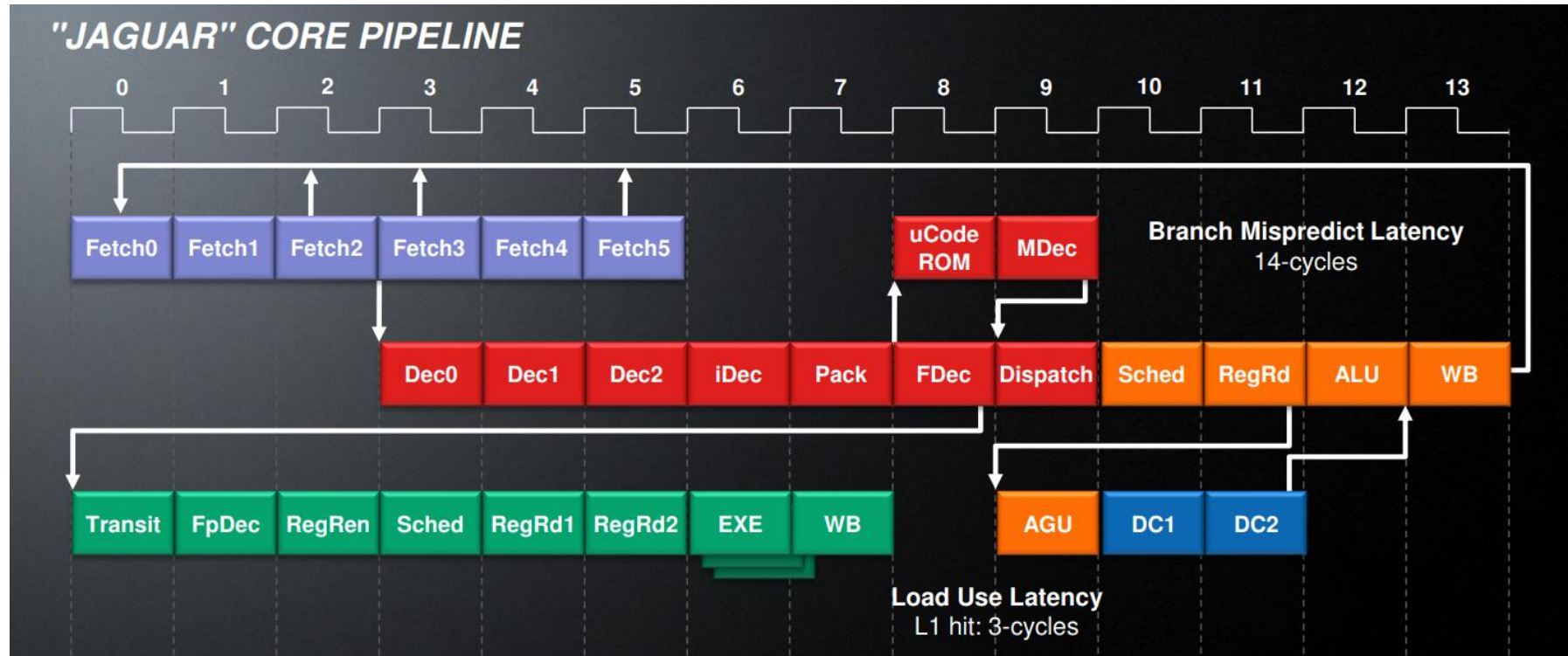
Sony/IBM/東芝 Cell (SPE)

Cell Broadband Engine Architecture and its first implementation—A performance view より



AMD JAGUAR

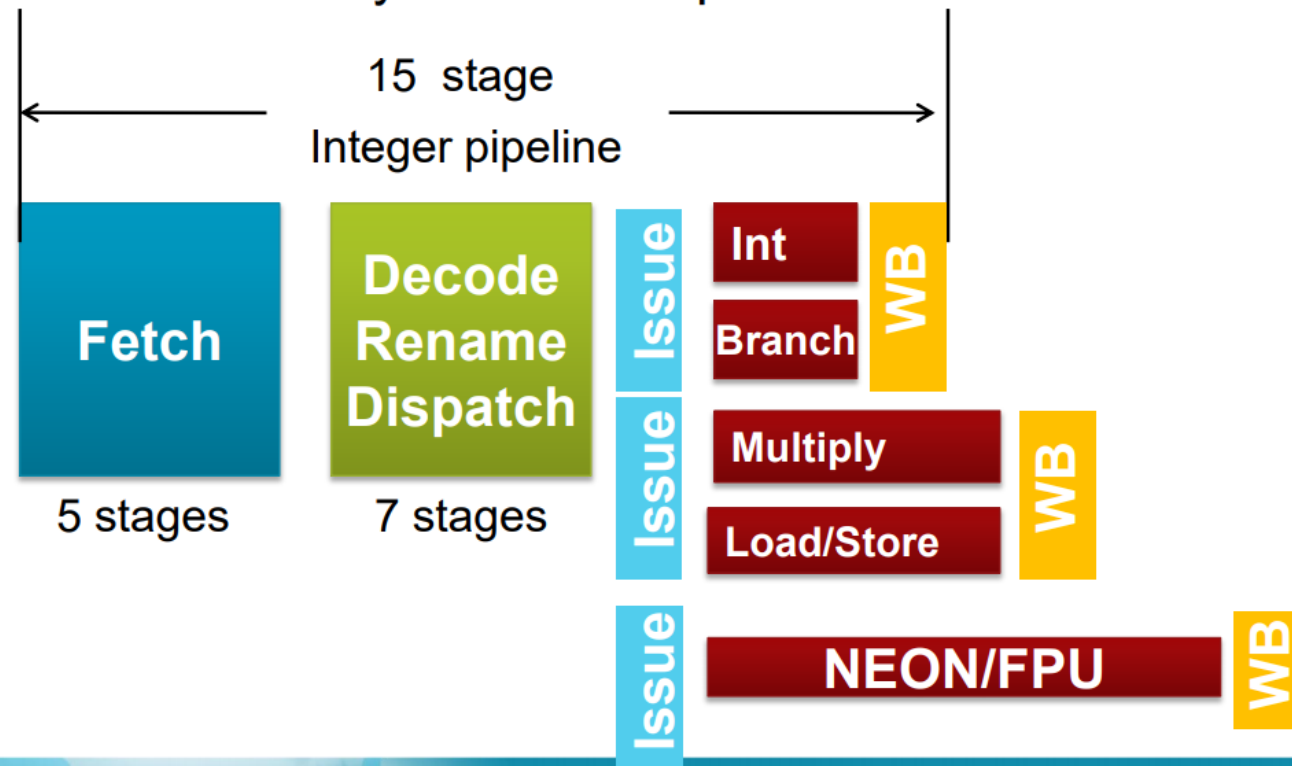
"JAGUAR" AMD's Next Generation Low Power x86 Core より



Cortex-A15 Pipeline Overview

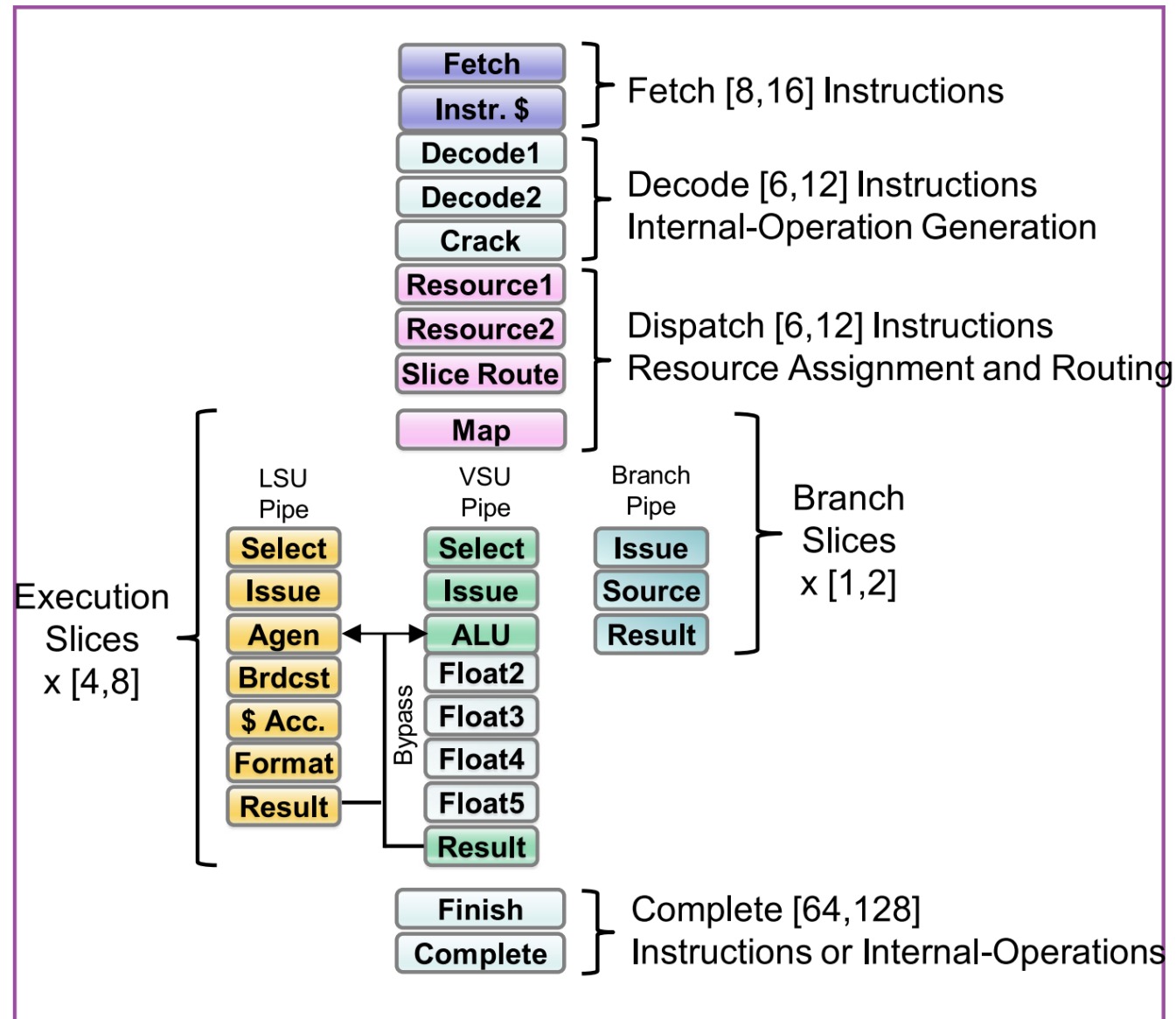
15-Stage Integer Pipeline

- 4 extra cycles for multiply, load/store
- 2-10 extra cycles for complex media instructions



IBM POWER9

IBM POWER9 processor core より



分岐予測の基本

用語の定義（1）

■ 方向分岐

- if 文のように, 2 方向に分岐する分岐命令

■ 間接分岐

- レジスタに格納されている値のアドレスに飛ぶ分岐命令
- 任意の場所に飛ぶことができる

用語の定義（２）

■ 分岐の成立/不成立

- 条件が成立（taken）： 指定されたアドレスへジャンプ
- 条件が不成立（untaken）： 次の命令（PC+ 4）に移る

■ 例： bne x1, x2, TARGET

- 成立： x1 と x2 の値が異なった場合は、TARGET にジャンプ
- 不成立： x1 と x2 の値が同じ場合は、次の PC に

用語の定義（3）

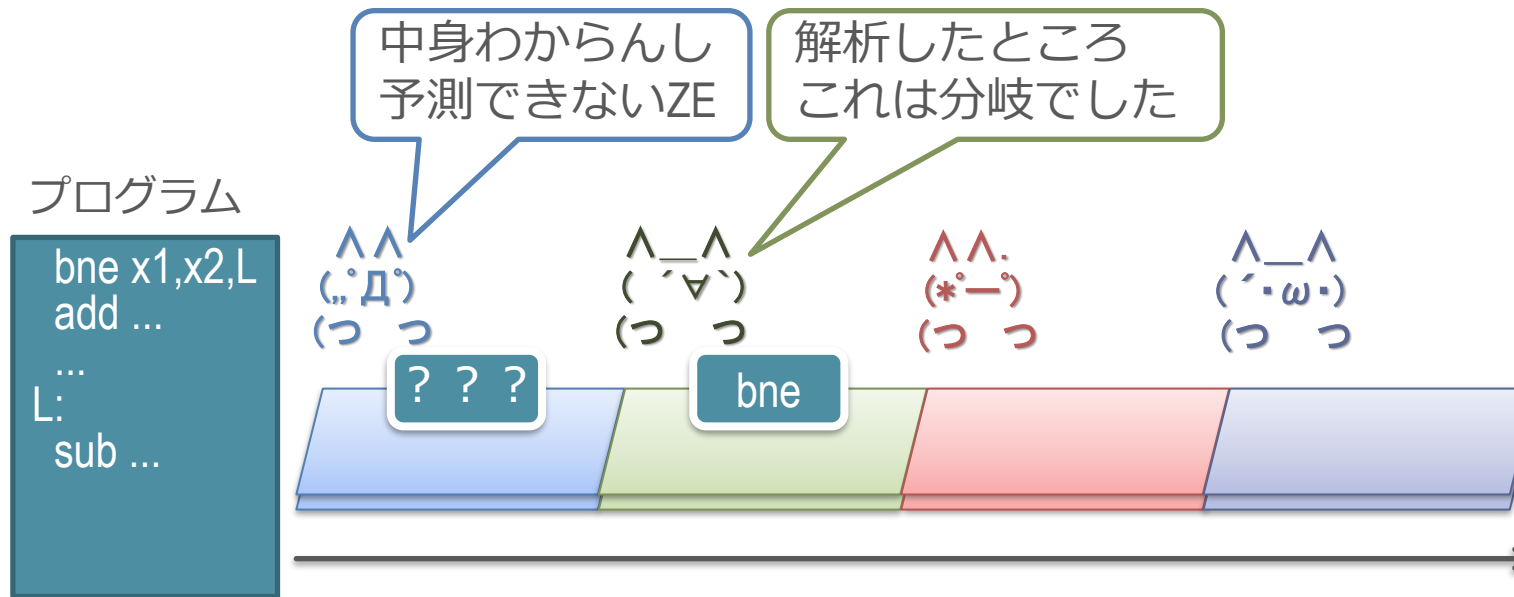
- 分岐先 アドレス or ターゲット
 - 分岐が成立した際の飛び先のアドレスのこと
- 前方分岐：
 - 分岐先ターゲットが分岐自身のアドレスよりも大きい分岐のこと
 - プログラムの進行方向に対して前方に飛ぶことから
- 後方分岐：
 - 分岐先ターゲットが分岐自身のアドレスよりも小さい分岐のこと
 - 後方に飛ぶ = ループを作る



分岐予測

- 分岐予測では、以下の3つを全て行う必要がある
 1. 分岐命令かどうか予測（分岐種別の予測）
 2. 分岐先ターゲット予測
 3. 分岐方向予測
- if-then-else の方向だけを予測していれば良いわけではない
- （今は方向分岐のみを扱い、間接分岐は考えない）

1. 分岐かどうか予測の必要性



- メモリから命令が取れるまでは、それが分岐かどうかはわからない
 - 命令フェッチは複数段にパイプライン化されていることが多い
 - 以降のターゲットや方向の予測をすべきかどうか、わからない
- 一方パイプライン先頭では即座に次のアドレスを予測しないといけない
 - 分岐かどうかわかるまでまっ待ちは、バブルができる

2. 分岐先ターゲットの予測の必要性

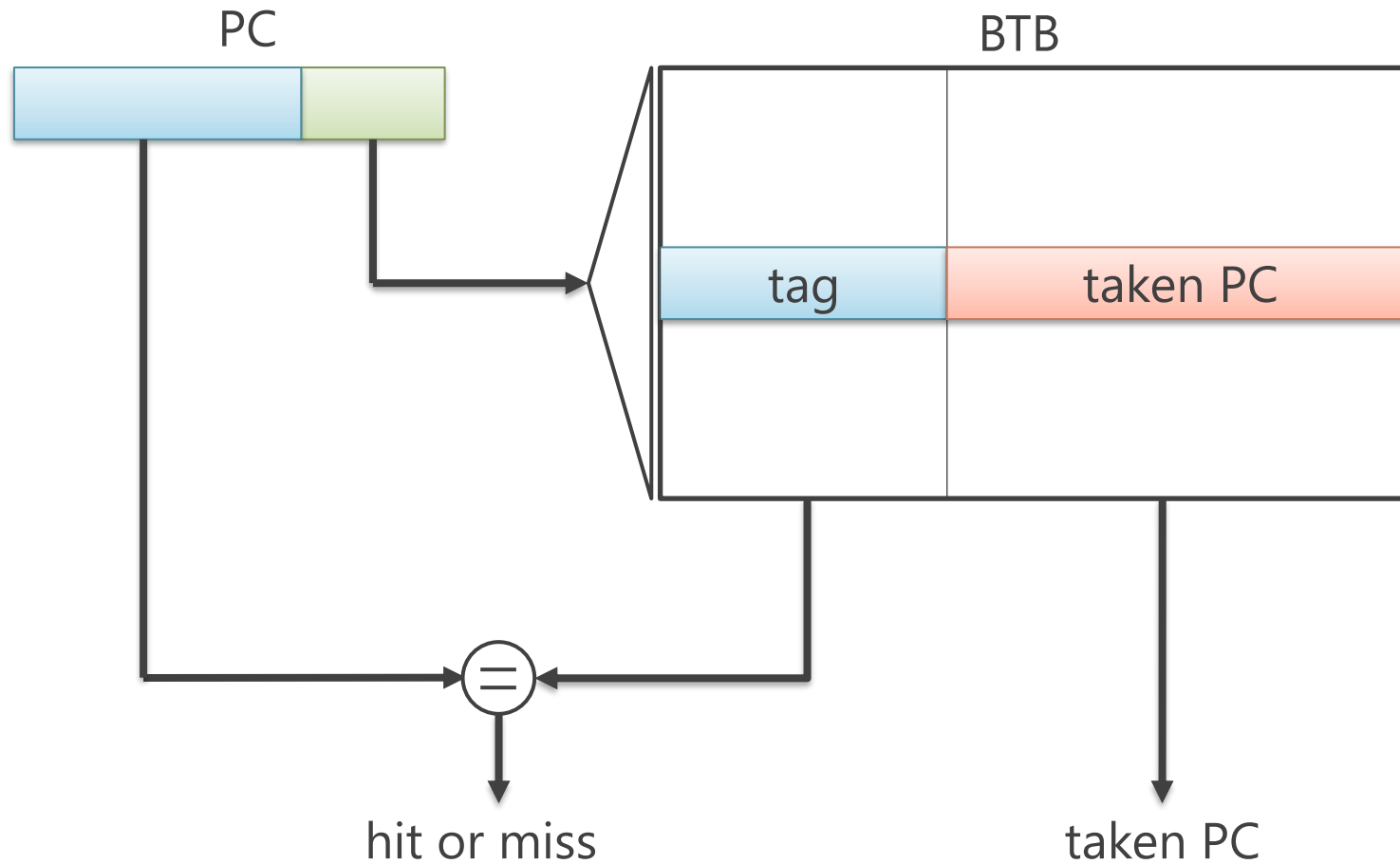


- メモリから命令が取れるまでは、分岐成立時の飛び先の場所もわからない
 - いくつ先 or いくつ前に飛ぶのか？

BTB（Branch Target Buffer）による予測

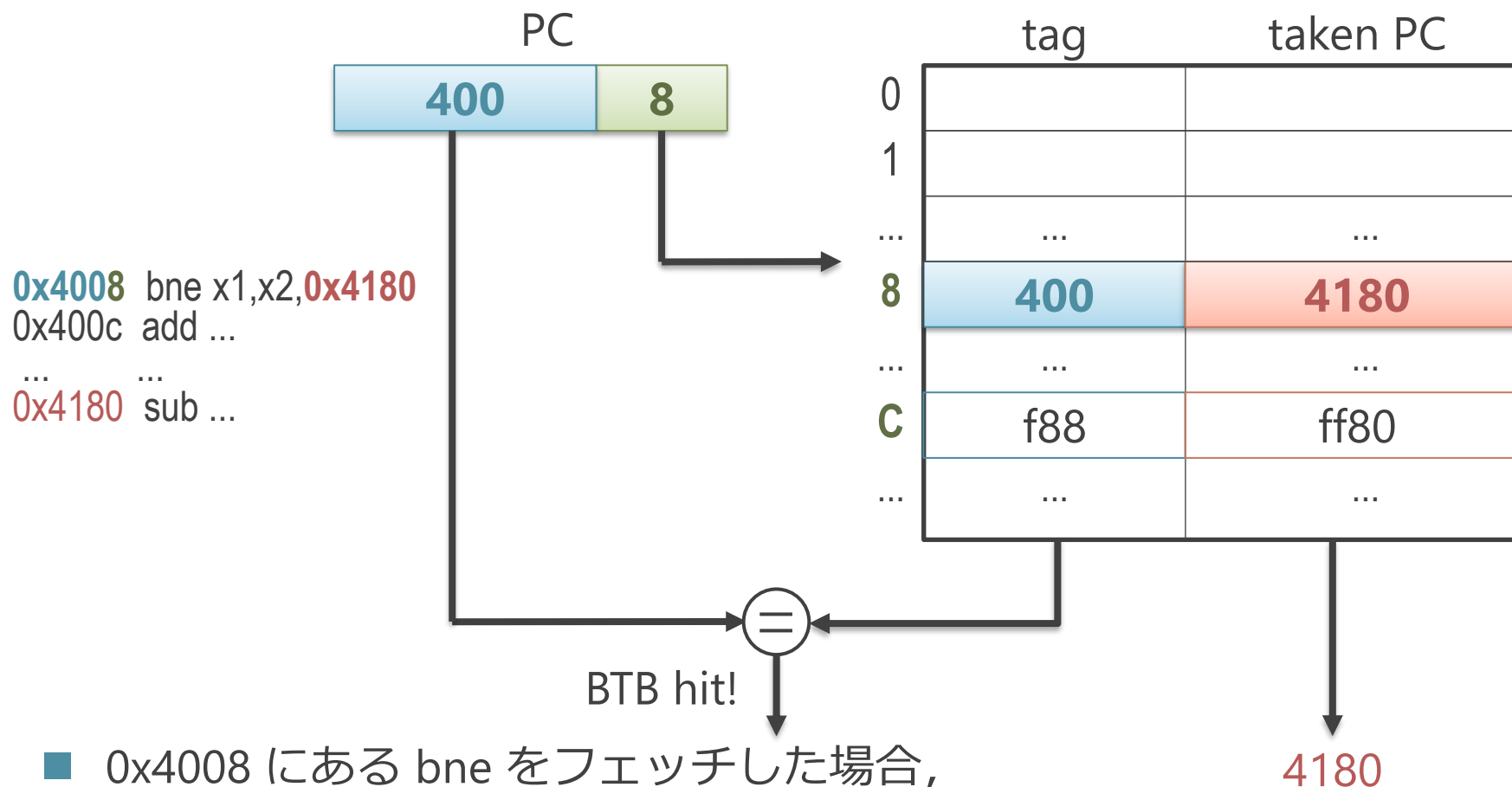
- BTB と呼ぶ表を使って以下を予測
 1. 分岐命令かどうか予測
 2. 分岐先ターゲット予測
- BTB
 - 入力：PC
 - 出力：
 - hit or miss
 - ターゲットのアドレス
- 分岐命令の実行時に、この表にターゲットを登録しておく
 - 次回からは、表をひくとターゲットがとれる

BTB (Branch Target Buffer) による予測



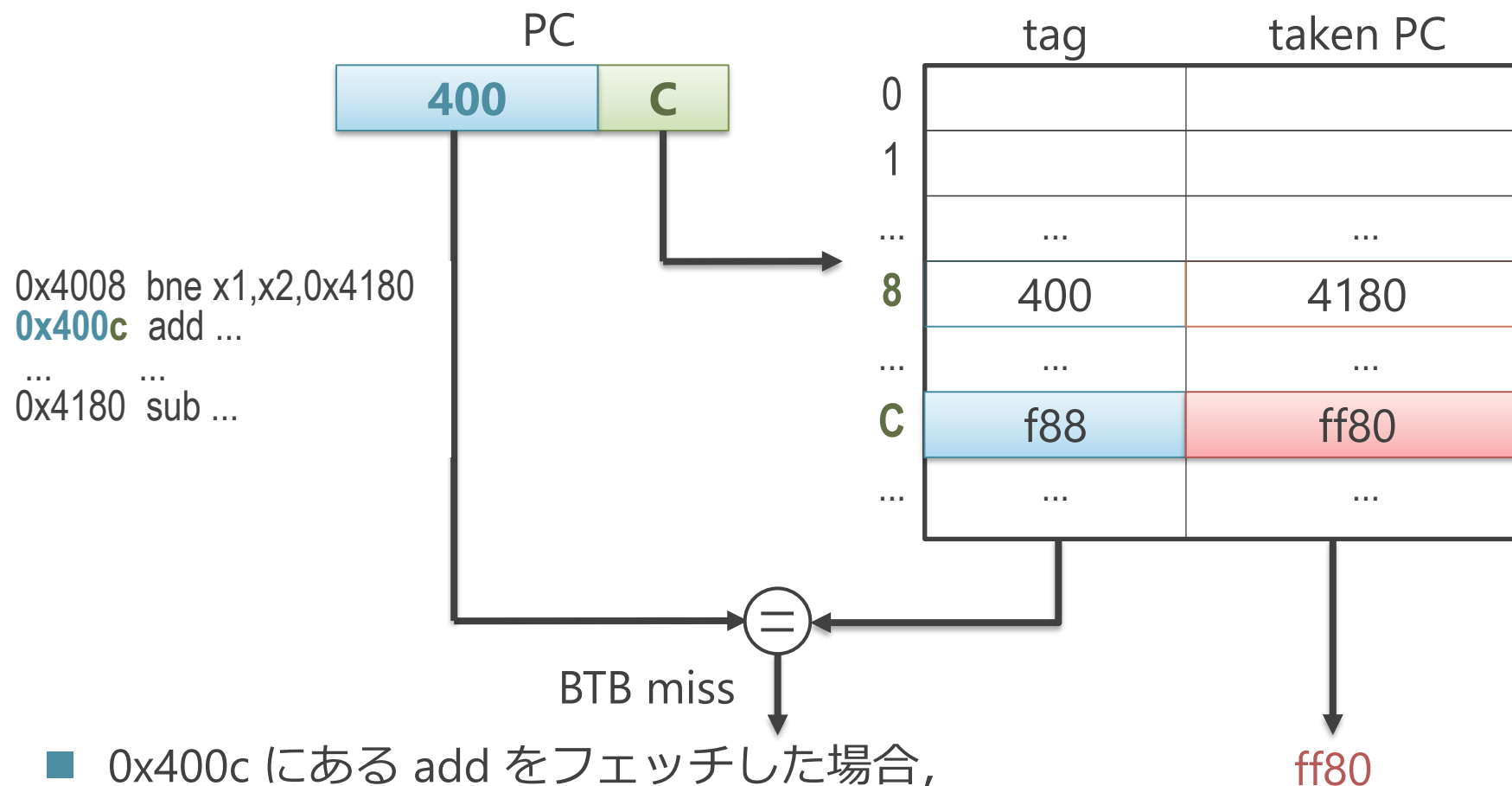
- 分岐かどうかと、分岐先ターゲットを同時に予測

BTB による予測（分岐命令の場合）



- 0x4008 にある bne をフェッチした場合,
 1. 0x4008 の下位の 8 を取り出し, BTB の 8 番エントリにアクセス
 2. 得られた tag と PC の上位の 0x400 が一致したのでヒット
 3. 0x4008 は分岐命令で, そのターゲットは 0x4180 と予測

BTB による予測（分岐以外の場合）



- 0x400c にある add をフェッチした場合,
 1. 0x400c の下位の c を取り出し, BTB の c 番エントリにアクセス
 2. 得られた tag と PC の上位が不一致なのでミス
 3. 0x400c は分岐命令ではないと予測

(古典的な) BTB の特徴

- 高速にアクセスする必要がある
 - 1 サイクル以内に処理が完結する必要がある
 - そうしないと、毎サイクル命令フェッチができない
- 1. エントリ数は比較的小さい
 - 最大でも数Kエントリ程度
 - 色々と方法が発明され、最近はそうでもなくなってきた（後述）
- 2. ハッシュ表としては、かなり単純な構造を持つ
 - ハッシュ関数は、アドレスの一部を切り出してそのまま使う
 - 表の各エントリは固定長（古いものは上書きされる）

- 階層化された BTB を持つ場合もある
 - AMD Zen : 16エントリL0+512エントリL1+7168エントリ BTB
 - ARM Cortex A72 : 64エントリL1 + 2KエントリL2 BTB
- L2 BTB アクセスには数サイクルかかる
 - L2 BTB により分岐が判明した場合, 予測ミスからの回復が行われる

最近の BTB

- 非常に大型化している
 - Zen4 : 1.5K エントリ L1 + 7Kエントリ L2
- 推定される要因：
 - デカップルド・フロントエンド (decoupled frontend)
 - 分岐予測器が命令キャッシュアクセスから分離されている構成
 - ◇ 分岐予測器がフェッチよりも先に走ることでプリフェッチする
 - 分岐予測精度や BTB ヒット率が重要なため, 大型化
 - アヘッド・パイプライン (ahead pipelining)
 - 1 サイクル前の PC で BTB を引き, タグ比較はそのサイクルの PC で行う
 - パイプライン化されるため, 1 サイクル内で処理が完結する必要がない
- デカップルド・フロントエンドやアヘッド・パイプラインは次回以降で述べる

分岐かどうか&分岐先ターゲット予測のまとめ

- 分岐かどうか & 分岐先ターゲットを予測する必要がある
 - 命令をデコードするまでは、それらがわからない
- BTB を使った予測
 - BTB：機能的にはハッシュ表
 - 入力：予測対象の PC
 - 中身：分岐先ターゲット
 - フェッチ時は、まず BTB にアクセスして分岐かどうかとターゲットを予測

- 以降, 「分岐予測」と言った場合は「分岐方向予測」の意味に
- 以下の2つに大きく分けられる
 1. 静的分岐予測
 2. 動的分岐予測

静的分岐と動的分岐

```
// 10回まわるループ
i1:      li  x1 ← 0           // x1 を 0 に初期化
        L:
i2:      add x1 ← x1 + 1      // x1 をインクリメント
i3:      bne x1 != 10, L      // x1 が 10 でなければ L に飛ぶ
```

■ 静的分岐：

- プログラム内に書かれている分岐命令のこと
- 上のコードでは、1つの静的分岐（i3）がある

■ 動的分岐：

- 実行中に現れる分岐命令のこと
- 上のコードが実行された場合、i3 は 10 回実行される
- = 10個の動的分岐がある

■ 同様に、静的命令や動的命令という場合もある

分岐方向予測

- 以下の2つに大きく分けられる

- 1. 静的分岐予測

- 静的分岐に対する予測
 - プログラム開始時に予測結果は決まっており, 実行中に予測結果は変化しない

- 2. 動的分岐予測

- 動的分岐に対する予測
 - プログラムの実行中に予測結果が変化する

分岐方向予測

■ 分岐予測

1. 静的分岐予測

1. 常に不成立と予測
2. 前方分岐を不成立/後方分岐を成立と予測
3. プロファイルによる予測

2. 動的分岐予測

1. 常に不成立と予測

- 今の PC に対し, 次の PC を常に読む
- あまり精度は良くない
 - 統計的に, 大体 70% ぐらいの分岐命令は成立する
 - したがって, 予測ヒット率は 30% ぐらい
- 最も単純で, 予測のために特に追加のハードを必要としない
 - 古い CPU では実際にこれを搭載していたものも結構ある

2. 前方分岐を不成立/後方分岐を成立と予測

後方分岐

```
// 10回まわるループ
i1:      li    x1 ← 0           // x1 を 0 に初期化
        L:
i2:      add   x1 ← x1 + 1      // x1 をインクリメント
i3:      bne   x1 != 10, L      // x1 が 10 でなければ L に飛ぶ
```

- 統計的に、後方分岐は成立することが多い
 - ループを構成することが多く、繰り返し実行される
 - 典型的には 80% 以上が成立
- 前方分岐を不成立/後方分岐を成立
 - 前方分岐はコストを重視して、常に不成立と予測
 - 後方分岐は常に成立と予測

3. プロファイルによる予測

■ 予測方法

1. 分岐方向のプロファイルをとる
 - 事前にプログラムを実行して、静的分岐の方向の統計をとる
 - 「このアドレスの分岐命令は、大概成立 or 不成立」
2. プロファイル結果に基づき、命令にヒントを埋め込む
 - 成立 or 不成立 の傾向を命令コードに埋め込んでおく
 - コンパイラにより行う
 - 命令セットのレベルで対応が必要
3. CPU は命令内に埋め込まれたヒントに基づき予測

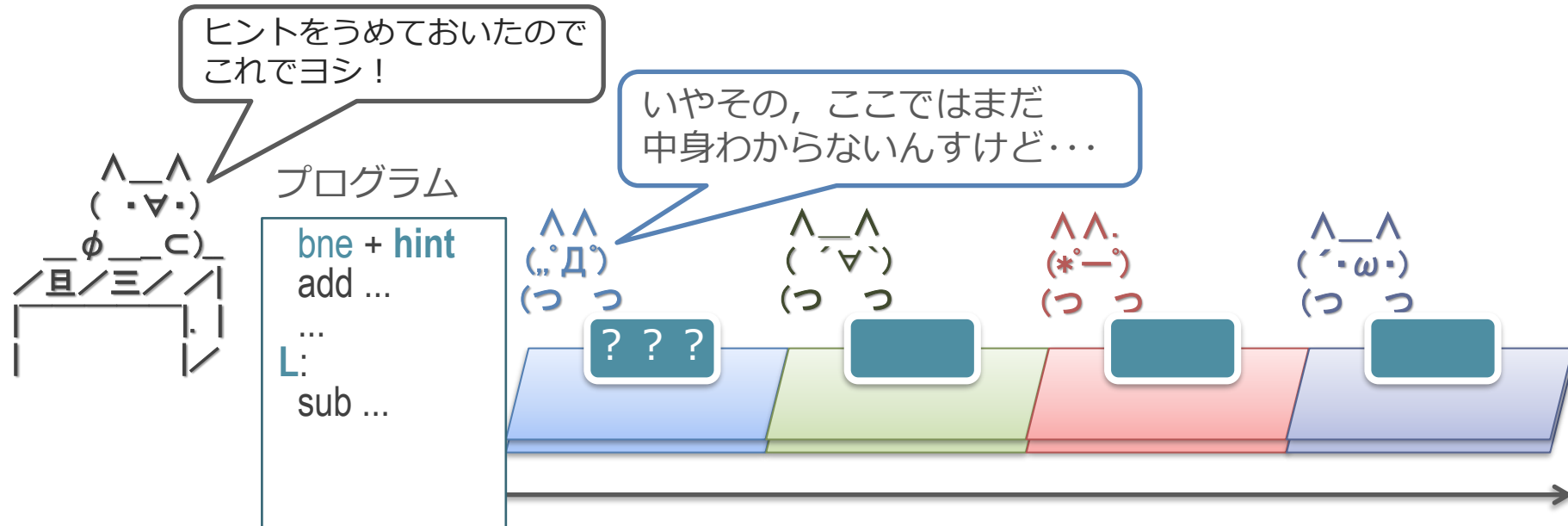
3. プロファイルによる予測

- そこそこの精度が出る
 - 静的分岐命令 1 つ 1 つの傾向が反映できる
 - 後方分岐だけど不成立が多い... とかに対応できる
 - 予測精度はだいたい 80% から 90% ぐらい

静的分岐予測の欠点

1. 分岐方向が毎回変わるようなものには本質的に対応できない
 - 例：同じ静的分岐で成立と不成立が交互に起きる
2. プロファイル時と挙動が異なる場合に対応出来ない
 - オプションや入力に応じてプログラムの挙動が大きく場合など
3. 意外とハードウェア・コストが安くない
 - 方向そのものの予測にはハードは必要がない
 - 成立すると予測する場合, BTB が別途いる
 - 分岐かどうか & 先ターゲット予測は必要
 - 「後方分岐かどうか」の予測や,
「成立/不成立のヒント」の予測を行う必要がある

「後方分岐かどうか」「成立/不成立のヒント」の予測



■ フェッチされた命令は、デコードするまでは以下がわからない

1. 分岐命令かどうか？
2. 分岐ターゲットはどこか？

■ 同様に、

- 「後方分岐かどうか」「成立/不成立のヒント」もわからない

別途ハードウェアが必要

- 「後方分岐かどうか」「成立/不成立のヒント」もわからない
 - 方向そのものを直接は予測しない
 - しかし, かわりに「後方分岐かどうか」等を予測する必要がある
- 別途それらを表に学習する？
 - 後述の動的分岐予測とあまりかわらない機構が必要

静的分岐予測のまとめ

- 静的な命令に対してあらかじめ予測
- 基本的に、今の CPU では使われていない
 - 予測精度の上限に限界がある
 - 意外とハードウェア・コストが安くない

分岐方向予測

■ 以下の2つに大きく分けられる

1. 静的分岐予測

- 静的分岐に対する予測
- プログラム開始時に予測結果は決まっており, 実行中に予測結果は変化しない

2. 動的分岐予測

- 動的分岐に対する予測
- プログラムの実行中に予測結果が変化する

動的分岐予測

- さまざまな予測手法を紹介

- 1. n ビット・カウンタ

- 1. 1ビット・カウンタ予測器
 - 2. 2ビット・カウンタ予測器

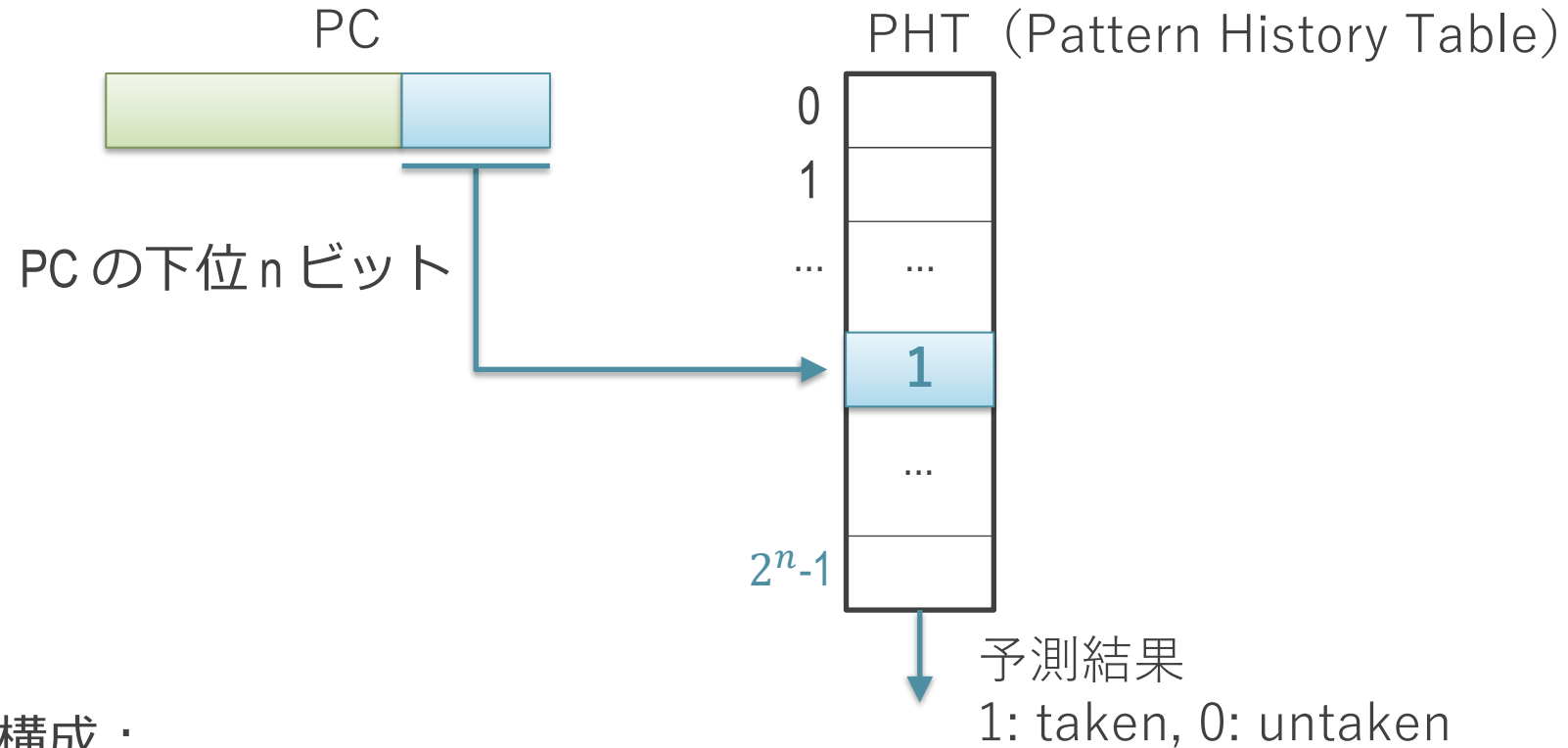
- 2. 履歴を用いたもの

- 1. ローカル履歴予測器
 - 2. グローバル履歴予測器
 - 3. より高度な予測器

- 下に行くほど先進的

- それぞれ上にあるものを下敷きとしているので, 順に説明

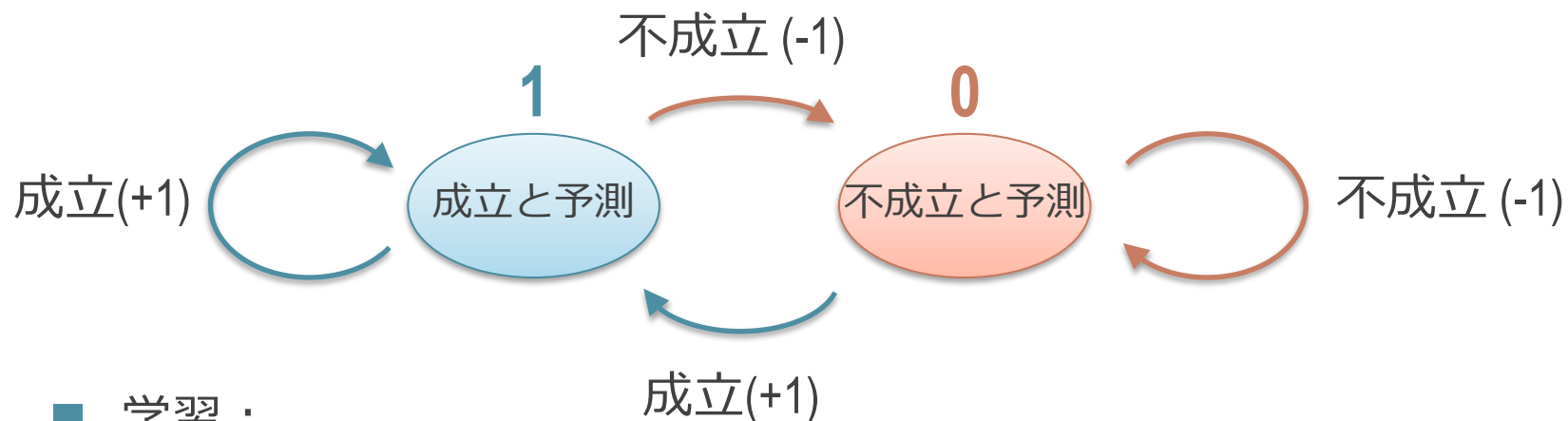
1ビット・カウンタ予測器



■ 構成 :

- PC の下位 n ビットをインデクスとしてアクセス
 - エントリ数は $2^n - 1$ エントリ
- 各エントリは 1 ビットの飽和型カウンタ

1ビットの飽和型カウンタの状態遷移図



■ 学習 :

- 分岐が成立したら +1, 不成立なら -1
- 1 を超えたら1, 0を下回ったら0

■ 予測 :

- 1 なら成立, 0 なら不成立

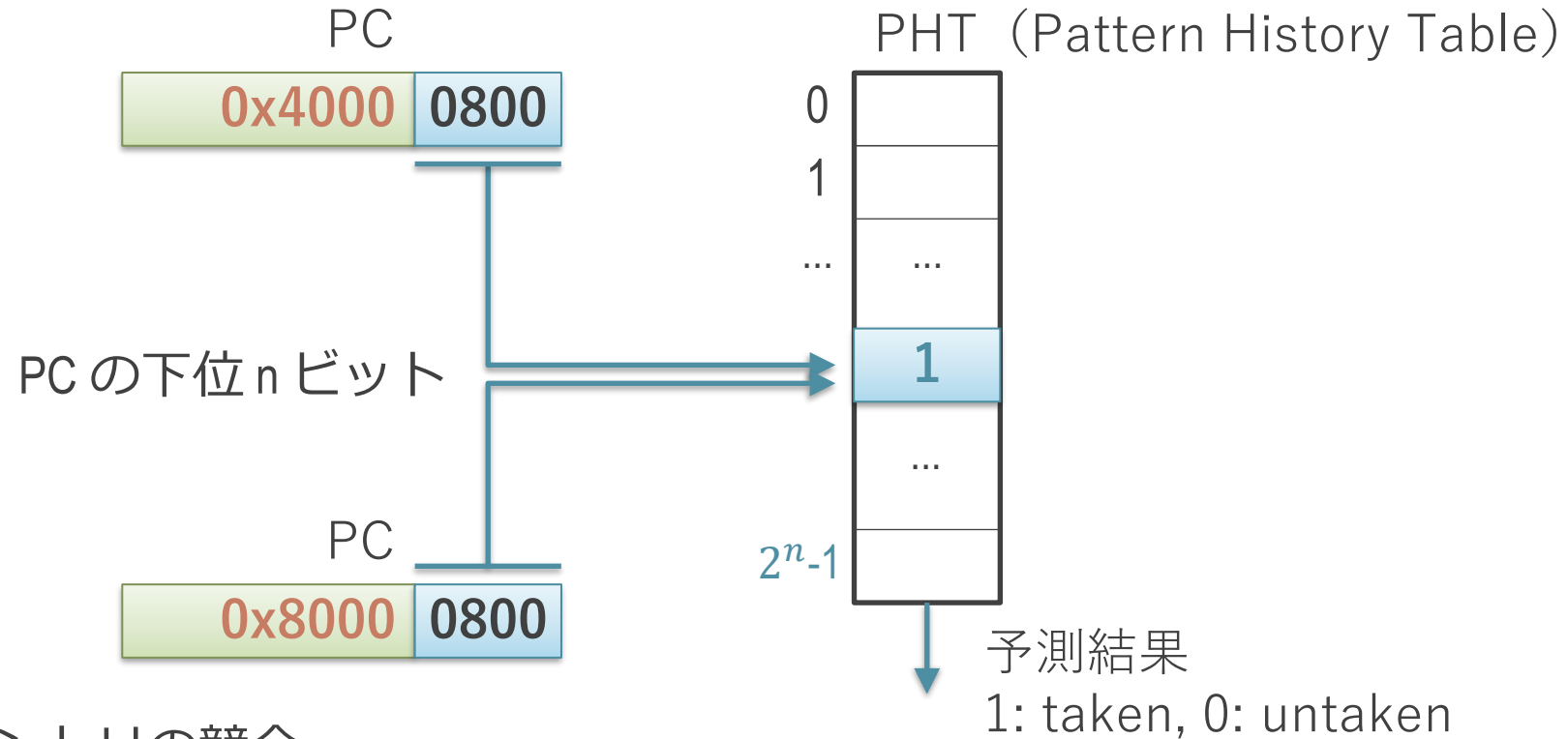
1ビット・カウンタ予測器の意味

- 動作：静的分岐のPCごとに，前回の動的分岐の結果を再現
 - 分岐が成立 → カウンタが1に → 次回は成立と予測
 - 分岐が不成立 → カウンタが0に → 次回は不成立と予測

1ビット・カウンタ予測器の利点

- 静的な分岐命令の分岐方向に偏りがある場合に有効に働く
 - 例：ソースコード～行目の if は大体こっちに行く
- 静的分岐予測では対応不能な場合にも対応出来る
 - 偏りはあっても、コンパイル時には決定できない場合
 - 例：コマンドライン・オプションによる分岐
 - -hoge の時はこの分岐は常に不成立
 - -fuga の時は常に成立

エントリの競合



■ エントリの競合

- 下位ビットがかぶると、異なるアドレスの分岐が同じエントリを使ってしまう
- 偏りが逆方向だと、予測精度を落とす
- エントリ数を増やす事で解消可能

分岐方向予測

1. 静的分岐予測
2. 動的分岐予測
 1. n ビット・カウンタ
 1. 1ビット・カウンタ予測器
 2. 2ビット・カウンタ予測器
 2. 履歴を用いたもの
 1. ローカル履歴予測器
 2. グローバル履歴予測器
 3. より高度な予測器

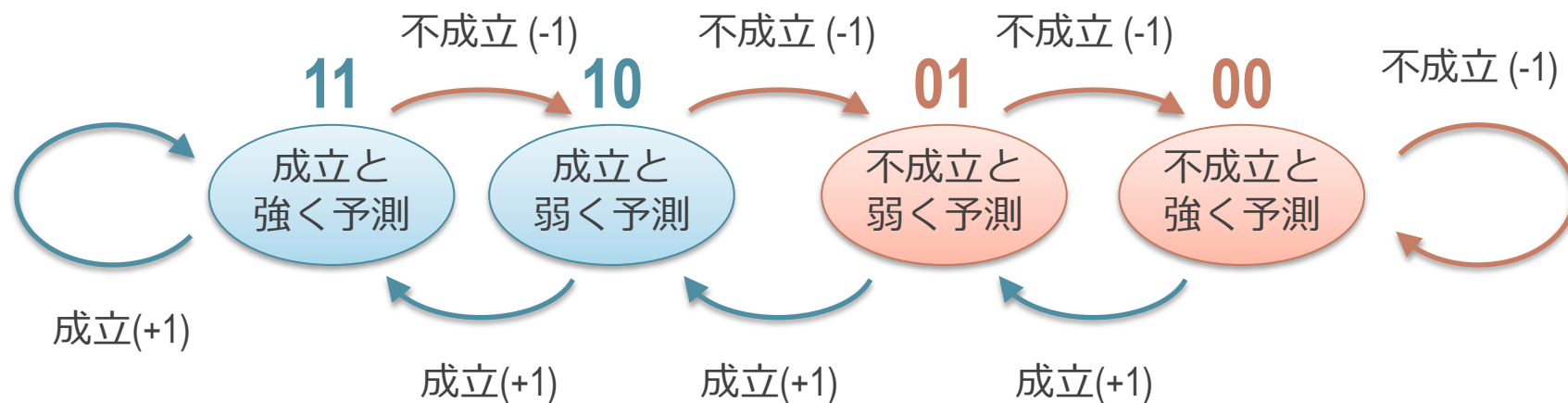
1ビット・カウンタ予測器の問題点：無駄な遷移

- 偏りがある場合に，無駄な状態遷移を起こす
 - たまに偏りと逆の方向に行った時に，戻ってくる時も必ず外す
 - 静的分岐予測で正しく偏りが拾えている場合，これは起きない
- 例： 成立：T，不成立：F とした場合，
 - カウンタ : 1 1 1 1 0 1 1 1 1 1 0 1 1 1 1
 - 予測 : T T T T F T T T T T F T T T T
 - 実際の方向 : T T T F T T T T T F T T T T T
 - （不成立は2回だが，予測は4回はずしている

2ビット・カウンタ予測器

- 1ビット・カウンタ予測器のカウンタを2ビットにする
 - 1回反対方向に行っても，次回も元の方角を予測することができる

2 ビットの飽和型カウンタの状態遷移図



■ 学習 :

- 分岐が成立したら +1, 不成立なら -1
- 11(2進)を超えたら11, 0を下回ったら 0

■ 予測 :

- 1 なら成立, 0 なら不成立

2ビット・カウンタ予測器の動作

- カウンタが 2 以上なら成立と予測，そうでなければ不成立と予測
- 例： 成立：T，不成立：F とした場合，
 - カウンタ : 11 11 11 10 11 11 11
(10進表記 : 3 3 3 2 3 3 3
 - 予測 : T T T T T T T
 - 実際の方向 : T T T F T T T
 - 不成立は 1 回であり，予測ミスも 1 回

予測精度とカウンタの幅

- 一般に, 1 ビット・カウンタ予測器より性能が高いと言われる
 - 実際に Intel Pentium, MIPS R10000 などの CPU に搭載
- カウンタのビット数を 3 ビット以上にすることはあまりない
 - 特に性能が向上しないことが知られている
 - 場合によっては, 精度が落ちる
 - 分岐の傾向が変わった時に, 学習結果の反映が遅れると言われている
 - TAGE やパーセプトロン系の予測ではまた話が異なる (次回以降)
 - バイアスを取りたい場合がある

まとめ

■ 分岐予測

1. 分岐命令かどうか予測
2. 分岐先ターゲット予測
3. 分岐方向予測
 1. 静的分岐予測
 2. 動的分岐予測

■ 高度な予測器の話題は来週以降に説明

出欠と感想

- 本日の講義でよくわかったところ, わからなかったところ, 質問, 感想などを UTOL の出席欄に書いてください
 - LMS の出席を設定するので, そこにお願いします
 - パスワード : arc
- 意見や内容へのリクエストもあったら書いてください
- UTOL の出席の締め切りは来週の講義開始までに設定してあります
 - 仕様上「遅刻」表示になりますが, 特に減点等しません
 - 来週の講義開始までは感想や質問などを受け付けます

- 以下のスプレッドシートの自分の席のところに学籍番号を書いてください
 - https://docs.google.com/spreadsheets/d/1Sufjhmcvn4_wlnaJ3sAv3Qf47gH4Q1Ku_dlsfGvo3B4/edit?usp=drive_link
 - 学生側から見た場合の配置で（画面上が教団側），席の対応する行と列のところに書いてください

質問や感想

質問とか回答など

- 昔からMulti Threading（特にHardwareの方で）ってよく聞きますが、イマイチどんなものなのかがよくわかりませんでした。色々調べてみた結果、昨今のCPUは同時MTが主流らしく、パイプラインの実行ユニット（ALUやFPU）の空き状況を見て、動かせる命令をスレッドの垣根を越えて超並列で実行させるとなっているらしいです。これでレイテンシーが高まるため、低遅延を求めるゲーマー界限であえてこういう機能を止めるっぽいですね。

- 自分もフォワーディング機構を持ったプロセッサを作ったことがあるのですが、性能が全然出ないどころかむしろ悪化してしまった記憶があるので、アーキテクチャやハードウェアのことをよく考えた設計をする必要があって難しい分野だなと思った記憶があります。

- multi threadでも遅延スロットなどの設計が施されているのか気になりました

- 制御ハザードの分岐予測について、一か八かで計算してみて間違ってたら戻るという行動は人間っぽくて面白いなと思った。

- マイクロコードについては学部の講義で学習しなかったもので非常に面白かった。比較的新しいRISCですらマイクロコードを用いているなら、もっと単純な命令しか使用しない命令セットを作れば良いのにと考えた。

- 投機実行で、ミスが判明した時のリカバリはどのように行われるのか気になりました。たとえば、メモリへのストアが投機実行によって行われて、後からそれが違うとなった時、元のデータにリカバーするためにストア命令によるメモリの書き潰し起きないように工夫する必要があるのか気になりました。

- しよぼんはしよぼんのアクションとかいう神ゲーのおかげで知名度がギリギリ保たれている気がします

- x86とかarmでは構造ハザードを命令をマイクロ命令することで修正するのは結構力技のように感じいずれ命令セットの大規模な改修が迫られるような気がするのですが以外とそのままでも問題ないのかが気になりました。

- 遅延スロットはもう少し評価されても良い気がする。欠点1は毎度仕様を変えればよくて、欠点2は一度に扱える命令数が増えればいい？

質問とか回答など

- futexなどのsystemcallの待ちやsleepなどはどう実装されるのでしょうか?pipeline的にはそこはbubbleになっているのでしょうか

質問とか回答など

- 最後の部分でマルチスレッディングの話が出てきたと思いますが、ちょうど今受講している別の講義(オペレーティングシステム)でも何回か前にマルチスレッドが登場していて、ようやく少しずつ他のレイヤとのつながりが見えてきたなあと感じました
- ただ、あちらは今日のスレッディングの内容をブラックボックス化して、linuxにはこういうスレッド化用モジュールが搭載されているんだよ、といった形で扱っていたので中でどうなっているかの説明はなかったですが...

質問とか回答など

- 創造の輪講でハーザードなどに関する研究について聞くことが多いが、今回の講義でなぜ発生するのか、どこが問題なのかを50%ぐらいは理解できた気がする。

- AMD Bulldozerの写真において、L1キャッシュの配置はコアに対して対称的なのに対して、下段のキャッシュはそうとは限らないことが気になりました。タイミング計算をどう整えているのでしょうか。
- 恐らくこの後にスーパースカラプロセッサについての解説があると思うのですが、実際の研究の時間軸はどうなっていたか気になります。

質問とか回答など

- 制御ハザードの分岐予測による投機実行を行うのは、前回の結果と同じ分岐のほうに賭ける、といったシンプルな分岐予測であっても
- ```
for (int i = 0; i < 100; i++) {
```
- ```
    // 主な処理
```
- ```
}
```
- といったコードではループから最初のループと最後のループ以外で分岐方向をあてられて98%の精度で予測に成功するかと思うと、とても有効なことを実感しました。



- フォワーディングの導入はバブルを消せる一方でクロック周波数を下げうるといふトレードオフの関係にあると思います。総合的な性能で見たとき、一般的にはフォワーディングを導入したほうが有利なのでしょうか？

- 分岐予測について、大規模なプロセッサでは大量の命令を取り消す可能性があるとのことでしたが、大規模な場合、両方の命令を並列で行ってみるとかいう方法はないのでしょうか。ただ、分岐予測がそれなりに当たることを考えたら性能が悪化しそうだなとなんともなく思っています。

- イラストや具体例が多くて非常に分かりやすかったです。命令パイプラインにとって条件分岐があると制御しにくい, という話だったと思います。これに関して, 例えばプログラミングにおいてearly returnをするとネストが深くなりにくいから命令パイプラインにとって優しい, ということがあったりしますか?あるいはその程度の差はコンパイラに吸収されてしまうのでしょうか。

# 質問とか回答など

- 
- 
- 
- \('・ω・')ノ くうるせえ、NOP命令ぶつけんぞ
- | /
- UU

- 分岐予測ペナルティが最新のスマートフォン用CPUでは数百命令までになるようなケースがあるというのは意外だったのと同時に、今後もこの数値が上がっていく可能性があるのか気になった。

# 質問とか回答など

- バンク競合によるパイプラインのストールとかも構造ハザードの一種なんですかね?
- あと、例にあった `ld_inc` っていうのはかなり気味が悪い命令というかパイプライン構造に逆らって処理されるような命令で色々と副作用がありそうな印象を持ちました。

- あと、例にあった `ld_inc` っていうのはかなり気味が悪い命令というかパイプライン構造に逆らって処理されるような命令で色々と副作用がありそうな印象を持ちました。

- マイクロコードへの分解は、x86などの普及によって仕方なく生まれた技術なのか、歴史上どのようなISAが普及していたとしても登場していたであろう技術なのか気になりました。